SECURITY · MIGRATION GUIDE

PHP Commerce: Moving Off Shared Hosting

Familiar is not the same as safe. What to move, where to, and in what order — after the October 2026 delistings and checkout hijack.

> **Familiar is not the same as safe.** Protecting customers now takes wholesale change, not patches — especially where personal data (PII), card data (PCI) or health data (PHI) is involved. The October 2026 checkout hijack ran through apps merchants had trusted for years. Shopify and Google have been raising these requirements since 2022. Most of the deadlines have passed; what remains is enforcement — delisting, blocked installs and lost ad eligibility.

Who this is for: merchants, agencies and app developers running WordPress and WooCommerce, Magento or Adobe Commerce, or PHP-backed Shopify apps — loyalty, upsell, product options, wishlists — on shared or single-VM hosting.

The question it answers: what to move, where to, and in what order, so that one breached plugin, one compromised vendor or one runtime flaw cannot reach every customer on the server.

The PHP runtime itself is covered in the companion article: patch for **php2shell** (CVE-2026-109618 and CVE-2026-109619) and keep the Cloudflare WAF rules in Block — Entitlement Security → php2shell. This article is about the architecture around it. For the owner and legal view of the same deadlines — what goes dark, and who carries the liability — see the Risk & Liability Brief: The 2026 Client-Side Cliff.

Vocabulary

TERM	MEANS	IS NOT
Shared VM	One server where many sites share an operating-system user, a database user and a filesystem	A private server with isolated apps
Private VM with Docker	One server the internet cannot connect to, with each application in its own sealed container, reached only through a Cloudflare Tunnel	Anything with an open inbound port
TLS	Transport Layer Security: the protocol that encrypts a connection and proves, with a certificate from a trusted certificate authority, that the server is who it claims to be. The successor to SSL; use version 1.2 or 1.3	A guarantee the server is safe — only that you reached the right one, privately
TLS transport	The sealed, tamper-evident envelope data travels in on every hop — shopper to edge, edge to server, server to Xano, server to database	Only the padlock in the browser
CAA	Certification Authority Authorization: a DNS record listing the certificate authorities allowed to issue TLS certificates for your domain (`issue` , `issuewild` , `iodef`). Every publicly trusted CA must check it before issuing; one record at the apex covers every subdomain. See CAA records	A certificate, or something browsers enforce. A CA that ignores it can still issue, so pair it with Certificate Transparency monitoring
iodef (CAA)	The reporting tag in a CAA record, from Incident Object Description Exchange Format (RFC 7970). It names where a certificate authority should report an issuance request it refused because of your CAA, as a `mailto:` address or an `https://` URL: `0 iodef "mailto:security@example.com"` . Flag `0` means a CA that doesn't support it ignores it. A refused request can be the first sign someone is trying to get a certificate for your domain	A guarantee or a monitor. CAs aren't required to send reports, and many don't. It says nothing about certificates a permitted CA issued; watch Certificate Transparency logs (crt.sh, or Cloudflare's Certificate Transparency Monitoring) for those. Use a mailbox that actually receives mail

TERM	MEANS	IS NOT
Encoding (Base64, CDATA)	Repackaging bytes so they survive a text channel. Base64 maps every 3 bytes to 4 safe characters, and base64url swaps `+/` for `-_` for URLs. It wraps Xano's encrypted values, every JWS and JWE segment, and Kubernetes Secret `data:` fields. SVG `CDATA` marks a block, such as a script, as text rather than markup. Rust keeps the two apart: bytes are `Vec<u8>`, text is a `String` that must be valid UTF-8, and decoding bad input returns an error instead of garbage	Encryption, or signing. Anyone can decode Base64, so a JWS payload or a Kubernetes Secret is readable by whoever can see it. Confidentiality takes encryption (Xano `encrypt`, JWE), and proof of origin takes a signature. CDATA scripts and base64 `data:` URIs in an SVG are an XSS route, so render SVGs without running scripts
Use-after-free (UAF)	A memory-safety bug in hand-managed languages such as C and C++: a program frees memory but keeps a pointer to it, the space is reused (often with attacker-controlled data), and the old pointer is used anyway. It is a leading cause of browser zero-days and of PHP interpreter CVEs such as the `unserialize()` family. Rust's ownership rules reject it at compile time outside `unsafe` code, which is why the gate in front of PHP is Rust compiled to Wasm	Something page JavaScript can cause. JavaScript is garbage-collected, so a script on a Webflow, WordPress or Liquid page can only trigger an engine bug, not create one; browser updates and sandboxing handle that. The JavaScript lookalike is "use after destroy": an HTMX swap or CMS load-more removes nodes that listeners still point to. Delegate events from a stable parent and re-initialise after each swap. A C library compiled to Wasm can still have a UAF inside its own memory
Chain of custody (and Rust ownership)	A provable record of who created an item, who held it and who changed it, at every step from origin to use. For applications (firmware, software, plugins) it is signed builds, an SBOM, published hashes and reviewed dependencies (`cargo vet`). For data (orders, consent, mandates) it is signed records, signed receipts and an append-only change log. Rust ownership is the same idea inside a running program: every value has exactly one owner, handing it on (a move) ends the old owner's access, and loans (borrows) are checked by the compiler before the code runs. That is why safe Rust can't have a use-after-free: no code keeps access to memory after its owner has released it	Transport security. TLS protects one connection while it is open; it proves nothing about an item once it is stored, or after the next hop. Data can travel over perfect TLS and arrive with no proof of origin. A nightly ERP or CSV import that silently overwrites consent is a broken chain, even over secure connections. Rust ownership covers memory in one program, not records moving between systems; those need signatures

TERM	MEANS	IS NOT
Sidecar	A small helper that runs beside each app and does one job — here, asking Xano `/auth/me` who the caller is and attaching a verified answer	The code-injection attack; it closes the hole where an attacker types someone else's customer ID
Real-time data binding	Prices, stock and points read live from the system that owns them	A copy refreshed by a nightly sync
AI task binding	An automated job limited to one tenant, one task and the tools that task needs	An agent holding a platform key
Client-side redirect	A script in the shopper's browser sending them to another domain	A 301 issued by the server
RuBAC	Rule-based access control: every request is checked against written rules — who is asking, for which tenant, on which record, which fields, at what time — before the database is touched	Role-based access alone (RBAC), where a role such as "admin" opens everything that role can see

The requirements are not new

SINCE	REQUIREMENT	WHAT IT DEMANDS	REFERENCE
Oct 1, 2022	Shopify protected customer data (API 2022-10)	Personal data redacted by default; apps apply for access and meet Level 1 and Level 2 data-protection requirements	Shopify
Mar 2022, mandatory Mar 31, 2025	PCI DSS v4.0, requirements 6.4.3 and 11.6.1	An authorised inventory of every script on payment pages, integrity checks, and alerts on unauthorised change — the control that catches a checkout hijack	BDO summary
Mar 2024	Google Consent Mode v2 for EEA traffic	Consent signals for ad storage, analytics, ad user data and ad personalisation	Google
Aug 26, 2024	Shopify Customer Account API	At least Level 1 protected-customer-data approval	Shopify
Apr 1, 2025	New public Shopify apps GraphQL-only; REST Admin API legacy	New App Store submissions cannot use REST	Shopify Partners
Jun 30, 2026	Shopify Scripts stop executing	Checkout logic must be Shopify Functions	Shopify changelog
Oct 16, 2026, 15:00 UTC	Shopify API version 2025-10 no longer accessible	Apps still on unsupported resources after an upgrade deadline are delisted	Shopify versioning

October 2026: delistings and the checkout hijack

About **126 apps** were pulled from the Shopify App Store between October 14 and 18, 89 of them on October 15. Most of that number is versioning, not attack: API 2025-10 stopped being accessible on October 16, and delistings are routine — LetsMetrix counted 512 between September 22 and October 21, about 17 a day. Shopify has published no count and named no apps.

Only a few are tied to the hijack. Between October 10 and 20, attackers changed vendor-side settings so that shoppers clicking Checkout were sent to a lookalike payment page. **Shopify's own checkout was not breached.** The code came from vendors' configuration APIs and internal tools, not from the app files Shopify hosts — which is the risk of a third-party server holding store access.

APP	STATUS	BASIS
Hulk Product Options	Hijack confirmed (Oct 10)	Developer statement — cside
Selleasy	Hijack confirmed (Oct 13)	Developer statement — LetsMetrix
Swym Wishlist Plus	Hijack confirmed (about Oct 14)	Unauthorised configuration change — LetsMetrix
Rebuy Engine — PHP app	Delisted; **not** checkout hijacking, self-reported	Statement by James Van Erck, co-founder and COO, on LinkedIn, 22 October 2026: on Oct 14, attackers found an unprotected sign-up path that skipped inbox verification, registered accounts on the company's own email domain, and inherited staff privileges on internal development stores. They read product settings and data-source rules for 11 merchant stores; no edits, no API keys or tokens exposed; merchants notified within 24 hours; contained within hours. Not independently verified
Privy; Boost AI Search & Filter	Delisted Oct 18; link to the hijack unconfirmed	Tracker listings, no vendor confirmation
17TRACK; Consentmo GDPR Compliance	Delisted; link unconfirmed	Tracker listings; Consentmo's rests on a single confidential source

Rebuy's account is the identity half of the same lesson. Nothing in the runtime failed: an email address on the company's domain was treated as proof of being staff, and staff accounts carried standing access to stores. Its remediation — privileges only for a named group, multi-factor authentication for every merchant, a third-party penetration test of the authentication services —

is the permissions model this article argues for: identity verified at the server, never inferred from what the caller typed, and access granted per task rather than held by default.

The same mistake is not unique to apps. Identity providers and the software built on them have shipped it too:

CASE	WHAT WAS TRUSTED	WHAT IT ALLOWED	REFERENCE
Microsoft Entra ID — nOAuth (first disclosed June 2023; still present in about 1 in 10 of 100+ tested SaaS apps in June 2025)	An unverified email claim used as the user's identifier	With only an attacker-controlled Entra tenant and the victim's email address, full takeover of the victim's SaaS account. MFA, conditional access and Zero Trust policies did not stop it	Semperis
Okta AD/LDAP delegated authentication (advisory Nov 1, 2024; introduced July 2024)	A cached earlier login for usernames longer than 52 characters — often email addresses	Sign-in with the username alone, without the right password, where MFA was off	Okta advisory
Rebuy Engine (Oct 14, 2026, self-reported)	An email address on the company's own domain, never verified	Staff-level reads on internal stores and settings reads on 11 merchant stores	Co-founder and COO, LinkedIn, 22 Oct 2026

The rule all three break is the same: **an email address is a contact detail, not an identity.** Identify a user by the identity provider's immutable subject — the OpenID Connect `sub` with its issuer, in Entra the object ID with the tenant ID — require a verified email before linking any account, restrict which tenants may sign in, and grant elevated access only to a named group. MFA is necessary and, as nOAuth shows, not sufficient.

Why `/auth/me` claims and extras refuse the impostor

In the CRM Sync model, nothing an app does is decided from the email address. Every request is resolved through `/auth/me`, which reads five things in a

fixed order, and each one is a row someone had to be granted rather than a fact inferred from what the caller typed:

STEP	WHAT `/AUTH/ME` READS	WHAT IT PROVES	WHAT IT DOES NOT ACCEPT
1 · Token	A server-signed session token, checked against a logout denylist on every read	Which subject this is — the token's subject ID, nothing more	An ID, email or role sent in the request body
2 · Account status	The subject's row, before any write runs	The account is live, not deleted or tombstoned	A valid token for a deleted account
3 · **Claims** (`user_claims`)	Identity links keyed by the provider's immutable subject — Google's `sub`, the Shopify customer ID — and `email_verified` as the provider asserted it, one row per subject	Which external identities belong to this subject	An email match as proof that two logins are the same person
4 · **Extras** (`user_extras`)	Profile and preference fields, cleared when marketing consent is withdrawn	What personal data exists, and that it is consented	Profile data held by the app that renders the page
5 · Entitlement	Permissions granted to the subject, scoped to a shop	What this subject may do, and where	Privilege inferred from an email domain, a staff flag or an account's age

Platform administration sits outside this path entirely: a separate root credential and revocable tenant-scoped tokens, never a user account with an elevated flag.

INCIDENT	WHAT THE ATTACKER HAD	WHERE `/AUTH/ME` STOPS IT
Rebuy	A new, unverified account on the company's own email domain	Step 5. A new subject has no entitlement rows, and no rule anywhere grants permissions by email domain — so the account authenticates and can reach nothing. Step 3 records it as unverified, and no claim links it to any staff subject
Entra nOAuth	An attacker's tenant asserting the victim's email address	Step 3. Subjects are matched on the provider's immutable subject, not the email claim, so a different tenant's identity is a different subject with no claims, extras or entitlement of the victim's
Okta delegated authentication	A username and a cached earlier sign-in	Step 1. Identity is only ever a server-signed token for a subject ID, checked against revocation on every read; there is no username-plus-cache path to a session

Closed on 22 October 2026. Writing this section surfaced the same mistake in CRM Sync's own code, and it is fixed in production. An email-only match now attaches a login to an existing subject only when the inbox is proven — the provider verified the address, a Shopify one-time-code login, or an invitation link — and an unverified login is refused rather than attached. A typed email on a public form writes nothing until a single-use link mailed to that address is clicked, and every unconfirmed submission gets the same reply, so the form cannot reveal whether an address has an account. Rewards enrolment takes identity only from a session. Four static checks in the release gate fail if any of this regresses. The full claims-and-extras model is in Entitlement Security → claims and extras.

Merchant checks: find these apps among those installed, delisted ones included; review app embeds and custom JavaScript fields for scripts nobody added; run a test checkout in a private window and confirm it lands on Shopify's checkout domain; read Shopify's notice for your store's exposure window. Shoppers who paid by Shop Pay or PayPal were generally not exposed; shoppers who typed a card number during a window were.

Remove every client-side redirect to another domain

The hijack worked because a script in the shopper's browser could send them somewhere else. Any script that can do that today is a hijack waiting for a compromised vendor — legitimate ones included: market redirectors, affiliate rewriters, "faster checkout" buttons, leftovers from a previous storefront.

REMOVE — CLIENT SIDE	REPLACE WITH — SERVER SIDE
`window.location` or `location.href` in theme code, app embeds or tag managers	Redirect rules at the Cloudflare edge or a Worker: a 301 to an allow-listed destination
`<meta http-equiv="refresh">` pointing off-domain	The same edge redirect, or a link the shopper chooses
Scripts that rewrite links or the checkout button	Checkout posts only to the store's own Shopify checkout
Geo or market redirectors in the browser	Market routing at the edge, on Shopify Markets domains or subfolders
Old domains forwarded by script	Domain-level 301s in one redirect table under version control

Then hold it: a destination allow list for every server redirect; a Content Security Policy with `script-src` limited to an inventoried list and `form-action` limited to the store's own origins and checkout; a payment-page script inventory with change alerts (PCI DSS 6.4.3, 11.6.1); and a private-window test checkout on every release.

What is being replaced

RISK ON A MAMP/WAMP-STYLE SHARED VM	WHAT HAPPENS	WHAT REPLACES IT
One OS user for every site	A file-read or upload bug in one plugin reads every site's `wp-config.php` or `env.php`	One container per site, read-only filesystem, no shared volumes
Public origin IP	Attackers skip the CDN and WAF	Origin reachable only through a Cloudflare Tunnel
TLS ends on the box, or not at all	Plain HTTP between CDN and origin	Edge certificate, Full (strict), origin certificate or tunnel
One database user	Every app can read every table	A role per service, plus Postgres row-level security
Admin panels on the open web	`/wp-admin`, Magento admin and phpMyAdmin face everyone	Cloudflare Access on every admin path
Frozen PHP versions	End-of-life PHP survives because upgrading breaks a neighbour	A pinned image per site, rebuilt and scanned each release
Secrets in files	Keys inside the web root	Secrets injected at runtime
No audit trail	FTP edits leave no record	Git commit, image digest and deploy tag per change

Desktop publish: how MAMP and WAMP sites reach production

MAMP and WAMP are desktop bundles — **M**ac or **W**indows + **A**pache + **M**ySQL (or MariaDB)

- **P**HP — for running WordPress or Magento on a laptop. The SQL database inside is an ordinary MySQL server. The risk is not the laptop; it is how the site usually gets published from it:

STEP IN THE COMMON WORKFLOW	WHAT ACTUALLY MOVES	THE EXPOSURE
Build on MAMP/WAMP	A local MySQL with default settings, often user `root` with password `root` or none	Fine on a laptop; dangerous once copied
Export the database (`mysqldump`, phpMyAdmin)	One `.sql` file holding every table: users, password hashes, orders, customer emails, and `wp_options` with plugin API keys	A full copy of personal data in Downloads, email or a shared drive
Upload by FTP or a file manager	Code plus the dump, often left in the web root	`backup.sql` or `site.zip` downloadable by anyone who guesses the name
Import on the server	The same `root`-style user, passwords and plugin keys	Desktop defaults become production credentials
Point `wp-config.php` at it	One database user with every right on every table	Any plugin flaw reads or changes everything
Repeat for each update	Production overwritten from the desktop, or the reverse	Orders and consent changes made live are lost or reset by the next push

WooCommerce and Magento make the dump worse. A store's database is not just content:

- **WooCommerce** keeps orders and customers in its own tables (the HPOS `wc_orders` tables on current versions, or `wp_posts` and `wp_postmeta` on older ones), payment gateway settings — including gateway secret keys — in `wp_options`, and REST API keys in `woocommerce_api_keys`, where the consumer secret is stored as written. A dump carries all of it.
- **Magento** stores database credentials and its encryption key (`crypt/key`) in `app/etc/env.php`, and encrypts sensitive configuration in `core_config_data` with that key. Copy the dump and `env.php` together — which a desktop publish usually does — and the encrypted values can be decrypted. It also holds customer password hashes, orders, carts and integration tokens, and needs

OpenSearch beside MySQL, which brings a second service and its own credentials onto the VM.

What replaces it:

1. **The desktop stays a sandbox,** with seed or synthetic data — never a copy of production customers. Anonymise anything realistic first.
2. **Publish code, not databases.** Code travels through Git and a built container image; nobody uploads a `.sql` file to production.
3. **Schema changes are migrations or versions,** applied in the image — WP-CLI and WooCommerce's own updater, Magento's declarative schema (`db_schema.xml`) with `bin/magento setup:upgrade` in production mode — or recorded by the data platform, not hand-run SQL.
4. **The production database has its own roles, made in its own ceremony:** no `root`, a user per service, no public port, TLS required.
5. **Desktop-to-production access goes through Cloudflare Access or a tunnel** — never an open MySQL port or FTP.
6. **Dumps are encrypted, expire, and live outside the web root,** deleted after the restore they were made for.

The migration path: MySQL on a VM to Postgres in Xano

Moving identity and customer data off WordPress's MySQL and into Xano's Postgres is a one-time migration, done once and carefully — not a recurring export. Xano provides the Postgres, containers and schema versioning; the rules on each endpoint are yours to write.

1. **Inventory and decide what moves.** List the tables (`wp_users`, `wp_usermeta`, WooCommerce orders and customers, plugin tables). Identity goes to Xano's users table and `user_claims`; profile and consent to `user_extras`; orders stay with Shopify if the store is moving there; products go to Shopify or the PIM. Anything nobody can name an owner for does not move.

2. **Do not carry password hashes.** WordPress hashes belong to WordPress. Customers sign in again through a magic link or a provider login, which creates a claim keyed by the provider's immutable subject — the step that stops email-match impostors.
3. **Export table by table, minimised.** One CSV per table with only the columns that move (`SELECT … INTO OUTFILE`, or `mysqldump --tab`), taken from a read-only copy, on an encrypted disk, never to the web root.
4. **Create typed tables in Xano first,** then load: Xano's CSV import for small tables, or a rule-checked endpoint (or Xano's bulk record functions) for large ones, so every row is validated and each field lands in the table that owns it. Schema versioning records who changed what and can roll back.
5. **Reconcile.** Compare row counts and a digest of each exported file against what Xano holds, and record the result in the ledger: which file, which digest, which rows were refused.
6. **Cut over behind the sidecar.** WordPress stops reading its own users table; the `/auth/me` sidecar answers who the caller is from Xano. The MySQL database goes read-only, then is retired after a full billing cycle.
7. **Close the old doors.** Delete every dump and CSV, drop the old database users, and rotate every key that ever sat in `wp_options` or `wp-config.php` — in a key ceremony, on a patched machine, one environment at a time.

WooCommerce and Magento: what lives where, and where it goes.

DATA	WOOCOMMERCE	MAGENTO	DESTINATION
Customer identity	`wp_users`, `wp_usermeta`, `wc_customer_lookup`	`customer_entity`, `customer_address_entity`	Xano users + `user_claims`; Shopify customer if the store moves
Password hashes	`wp_users.user_pass`	`customer_entity.password_hash`	**Not migrated** — sign in again by magic link or provider
Consent and preferences	Usually plugin tables or `wp_usermeta`	Newsletter tables, custom attributes	`user_extras`, as consent records with time and policy version
Orders and carts	HPOS `wc_orders` tables (or `wp_posts`), sessions	`sales_order`, `quote`	Shopify, or a read-only archive; never re-imported live
Products and stock	`wp_posts` + `wp_postmeta`, product lookup tables	EAV `catalog_product_*` tables	Shopify or the PIM, through the API
Payment and API secrets	Gateway keys in `wp_options`; `woocommerce_api_keys`; webhook secrets	`core_config_data`, encrypted with `crypt/key`; `oauth_token` integrations	**Not migrated** — revoke and reissue at each provider

Platform-specific steps:

- **WooCommerce:** revoke every REST API key and webhook, then rotate the gateway keys at Stripe, PayPal or the other provider — the copies in `wp_options` and every dump are now old.
- **Magento:** rotate the encryption key with `bin/magento encryption:key:change` (2.4.7 and later re-encrypt stored values) before the final cutover, then rotate payment and integration credentials at each provider. Keep the old `env.php` off every disk once the database is retired.

- **Both:** the PIM, DAM and import plugins attached to the store hold their own keys — Cloudinary, Akeneo, feed services. Rotate those too, as the CSV section describes.

After the move, the same rule from the CSV section holds: data reaches Postgres only through an endpoint that authenticates the caller and checks the row, never as a file loaded into the database.

Not just WordPress: PIMs, DAMs and the CSVs that feed PHP

The exposure in this article is not limited to the storefront. Product information (PIM) and digital asset (DAM) systems either run on PHP themselves or hand their data to PHP through plugins and CSV imports. Wherever data lands in PHP, it inherits the shared-VM risks above.

What the main systems are built on. Self-hosted open-source tools are mostly PHP on MySQL; enterprise SaaS mostly is not. The SaaS rows are drawn from public job postings and vendor documentation, not published architecture, so treat them as indicative.

PRODUCT	TYPE	BUILT ON	DATABASE	PHP IN THE CORE?
Akeneo	PIM	PHP (Symfony)	MySQL + Elasticsearch/OpenSearch	**Yes**
Pimcore	PIM + DAM	PHP (Symfony)	MySQL/MariaDB, optional OpenSearch	**Yes**
ResourceSpace	DAM, open source	PHP	MySQL	**Yes**
Adobe AEM Assets	DAM	Java	Apache Oak content repository	No
Stibo STEP, Informatica Product 360	PIM/MDM	Java	Oracle or SQL Server	No
inRiver, Aprimo, Sitecore Content Hub	PIM/DAM	.NET	SQL Server / Azure	No
Salsify	PIM	Ruby, with Python and Go services	PostgreSQL, Redis	No
Bynder	DAM	Python, Java and Scala microservices	—	No (PHP only on its marketing site)
Cloudinary	DAM / media	Ruby on Rails, partly moving to Go; C++ and FFmpeg/ImageMagick for media	—	No

SaaS without PHP still reaches PHP. Cloudinary has no PHP in its platform, but its PHP SDK, WordPress plugin and Magento extension do — and they run on the merchant's server. To sign uploads, they keep the Cloudinary **API secret** where PHP can read it: `wp_options`, `env.php` or a `.env` file. A php2shell-style flaw on a shared VM reads that secret, and with it an attacker can upload, replace or delete every image in the account — swapping product images, or planting files

on the brand's own media domain. The same is true of every PIM connector, feed plugin and DAM sync module installed into WooCommerce or Magento.

CSV and spreadsheet imports are a door into PHP

Even when the PIM is not PHP, the data often reaches the store as a CSV or spreadsheet loaded by a PHP importer — a WooCommerce or Magento import, a PIM connector, a nightly cron job. Each import is code reading a file someone else wrote:

RISK	WHAT HAPPENS	EXAMPLE
XML in spreadsheets (XXE)	An XLSX file is zipped XML; a crafted one makes the parser read local files — including `wp-config.php` or `env.php`	PhpSpreadsheet CVE-2024-45048, CVE-2024-47873 and CVE-2024-48917; Pimcore's data importer and admin UI were affected through it (GHSA-hq76-662x-7mw4)
Image URL columns (SSRF)	The importer fetches whatever URL a row names, from inside the network — cloud metadata endpoints and admin ports included	Any import that downloads `image_url` without an allow-list
Uploads in the web root	A "media" file that is really PHP is saved where the web server will run it	DAM or import folders inside `/wp-content/uploads` or `pub/media` on a shared VM
Formula injection	A cell beginning with `=`, `+`, `-` or `@` runs as a formula when staff open an export in Excel	Customer names or reviews exported to CSV
Full-file overwrites	A full upload rewrites fields the file should not own — prices, stock, consent	A nightly catalog file that resets every row it touches
Shared credentials	The import job runs as the same OS and database user as the store, often with an admin token in a crontab	One cron line exposes the whole database

What replaces it. The same pattern as the rest of this article: the file is checked before anything PHP-shaped touches it.

1. **Parse at the edge, not on the VM.** A Worker receives the file, validates every row against a typed schema, and writes through the platform's API (Shopify GraphQL, the Merchant API) — the PHP importer is retired, not patched.
2. **Field ownership.** Each source may write only the fields it owns. A catalog file never writes consent; an ERP file never writes reviews. Rows that try are refused and recorded.
3. **No fetching from rows.** Image URLs are fetched only from allow-listed hosts, outside the network, and stored in object storage (R2 or the DAM), never the web root.
4. **Sign the file.** The sender's digest and identity travel with the file, and each import writes a ledger row — so "which file set this price on 1 March?" is a query, as the AI Trust Framework Requirements asks.
5. **Until the importer is retired:** PhpSpreadsheet on a fixed release (2.2.1 or later for CVE-2024-45048, and the 1.9.4 / 2.1.3 / 2.3.2 / 3.4.0 lines for CVE-2024-47873), Pimcore and its data importer on their fixed releases, uploads outside the web root with PHP execution disabled there, and every DAM or PIM secret rotated when the site leaves the shared VM.
6. **Escape exports.** Prefix any cell that begins with `=`, `+`, `-` or `@` before a CSV leaves the system.

Three ways to load a product file: Matrixify, a bare database, and Xano with rules

The same `product.csv` can reach a catalog three ways. What differs is who checks it, and what a bad row can reach.

- **Matrixify** is a hosted Shopify app. It reads the spreadsheet on its own servers and writes through the Shopify Admin API under the scopes the store granted it.
- **A bare database load** is the file loaded straight into MySQL or Postgres — `LOAD DATA INFILE`, `COPY`, a phpMyAdmin import, or a PHP script reading

`product.csv` from an upload or FTP folder.

- **Xano with rule-based permissions** puts Postgres behind API endpoints. Nothing connects to the database directly; every write goes through an endpoint that authenticates the caller, checks typed inputs, and applies RuBAC rules before a row changes.

	MATRIXIFY CSV	BARE DATABASE / `PRODUCT.CSV`	XANO-HARDENED POSTGRES (RUBAC)
Who is checked	The store, once, when the app was installed	Whoever holds the database password or the folder	Every request: a signed session or scoped key, resolved by `/auth/me`
What a file can write	Everything the app's scopes allow — usually all products, customers and orders	Every table the database user can reach; often all of them	Only the fields the endpoint accepts, for the caller's tenant
Per-row rules	None of yours — any row the sheet names is updated	None; MySQL has no row-level security	Rules on tenant, owner, record state and field: a catalog file cannot touch consent, prices outside a range, or another tenant
Bad or old file	Re-importing last month's sheet silently resets consent, prices and stock	Same, plus a malformed file can corrupt or truncate tables	Refused rows are rejected and recorded (requirement R8); fields the source does not own are ignored
Parsing exposure	Matrixify's parser, on Matrixify's servers	Your PHP parser, on your server — the XXE and upload risks above	Parsing at the edge (a Worker) into typed JSON; Postgres never sees the raw file
Transport	TLS from Matrixify to Shopify	Often none inside the VM; a database port sometimes open to the internet	TLS on every hop — browser to edge, edge to Xano, Xano to Postgres
Record of change	The app's job log	Usually none	A ledger row per write: who, which file digest, which rule allowed or refused it
Blast radius of a leaked credential	The whole store, until the app is removed	The whole database	One tenant's allowed fields, until the key is rotated

How the Xano side is hardened:

1. **No direct database access** in production — no public Postgres port, and any direct database connector off or read-only.
2. **Authentication on every endpoint**, with identity resolved server-side by `/auth/me`, never taken from a field in the file.
3. **Typed inputs.** Each endpoint declares the fields and types it accepts; anything else is dropped before the query.
4. **Rules before writes.** A precondition checks tenant, role and field ownership for every row, and filters every query by the caller's tenant — the API-level equivalent of row-level security.
5. **Separate keys per job and environment.** An import key can write products for one tenant; it cannot read customers or touch another workspace.
6. **Sensitive fields encrypted** with a key made for that deployment, so even a full export reveals ciphertext.

Choosing. Matrixify is a sound tool for a one-off migration on Shopify — remove it, or cut its scopes, when the job ends, and strip columns you do not mean to change. A bare database load should not exist in production. For a recurring feed from a PIM, ERP or supplier, the rules belong in front of the database, where every row is checked and every refusal is evidence.

The options, and what each one isolates

OPTION	TLS AND ORIGIN	CODE ISOLATION	ADMIN LOGIN	DATA ISOLATION	SCALING
Shared VM (today)	TLS on the box or none; public IP	None	Passwords on the open web	One database user	Up only
WordPress or Magento in Docker behind Cloudflare	Edge TLS, Full (strict); tunnel; no inbound ports	Container per app	Cloudflare Access	Role per service	One VM, up
EmDash on Cloudflare Workers	Edge TLS; no origin	Isolate per plugin, permissions manifest	Passkeys; SSO	Plugins have no direct database access	Pay per use
11ty + Decap CMS on Cloudflare	Edge TLS; no origin	No code runs per request	Access on `/admin` + GitHub OAuth	Static files; live data through APIs	Static requests free and unlimited
Jekyll on GitHub Pages	GitHub-managed TLS	No code runs per request	GitHub accounts	Static files	Free within 1 GB and a 100 GB/month soft limit; **not permitted for e-commerce**
Shopify Storefront Web Components on Webflow, WordPress, Astro/EmDash, 11ty or AEM	The site's own TLS edge; checkout on Shopify	No commerce code on the site — Shopify's script renders products and cart	The site's own; customer accounts on Shopify	The site stores no customers, orders or cards	Scales with the site; commerce scales on Shopify

OPTION	TLS AND ORIGIN	CODE ISOLATION	ADMIN LOGIN	DATA ISOLATION	SCALING
Shopify Functions in Rust	Runs inside Shopify	WebAssembly sandbox, declared input only	Shopify app permissions	Scoped to the shop	Shopify scales it
Kubernetes + Postgres RLS	Edge TLS; NetworkPolicy; TLS to the database	Pod and namespace	Access + SSO	Row-level security per tenant and actor	Horizontal; highest cost

Order of moves: static first, for every page that needs no login. EmDash next, where editors need a live CMS. WordPress or Magento in Docker last, only for what cannot yet leave PHP.

EmDash was announced by Cloudflare on April 1, 2026 as a v0.1.0 developer preview — MIT licensed, TypeScript on Astro, no WordPress code. Content imports from a WXR export or the EmDash Exporter plugin; themes must be ported. It has no commerce, so WooCommerce stays in Docker or the store moves to Shopify. Pilot it on a blog subdomain first. Cloudflare: EmDash

The static path has run in production for years — Jekyll and GitHub Pages since 2008, 11ty since 2017, Decap since 2016 as Netlify CMS. GitHub Pages' terms forbid running an online business or e-commerce site on it, so a store built with Jekyll or 11ty is hosted on Cloudflare.

One machine, isolated: a private VM with Cloudflare and Xano, versus shared hosting

WordPress does not need a cluster to be isolated. One private VM — WordPress in its own container, reachable only through a Cloudflare Tunnel, behind the Cloudflare WAF, with identity and data on Xano — closes the gaps that shared and entry-level managed hosting leave open. The difference is not the WAF itself; it is **whether the WAF can be the only way in.**

	PRIVATE VM: DOCKER + TUNNEL + WAF + XANO	SHARED HOSTING (E.G. GODADDY, HOSTINGER ENTRY PL
Neighbours	None — your containers only	Many customer accounts on one server and kernel
Isolation	A container per app, read-only filesystem, no shared volumes	Account-level: Hostinger states each account runs in a CloudLinux LVE container, which limits resources and separates accounts
Can the origin refuse everything but Cloudflare?	Yes — no inbound ports; `cloudflared` makes outbound connections only	Generally no: running `cloudflared` or a firewall neec root, which shared plans do not give
WAF	Your Cloudflare WAF: managed rules, OWASP set, custom rules and rate limits you control	You can proxy DNS through Cloudflare, but the origin IF stays reachable, so attackers can go around the WAF
Admin login	Cloudflare Access in front of `/wp-admin`; nothing to brute-force	Passwords on the open web
Identity and data	Xano `/auth/me` resolves the caller; Postgres behind rule-checked endpoints	WordPress users table in a MySQL shared with the account's other sites
Secrets	Injected at runtime; signing keys never on the VM	In `wp-config.php` and `wp_options`, readable by a PHP on the account
Patching PHP	You pin and rebuild the image	The host's schedule

What this means. Cloudflare's WAF can be engaged securely only when it is the sole path to the origin. On a private VM that is a property of the network — no inbound port exists. On shared hosting it is a hope: the WAF filters the traffic that comes through Cloudflare, and nothing stops traffic that goes to the server's address directly. Managed hosts like WP Engine close much of that gap with their own Cloudflare-powered edge, but the rules, keys and data model stay theirs, and the site's identity still lives in a WordPress users table.

RuBAC by design: claims, extras, configuration and EdDSA keys

Rule-based access control is only as strong as the facts the rules read. In this architecture each fact comes from a place the WordPress container cannot write, so a compromised plugin can render a page but cannot change who the caller is, what they consented to, or what the rules say.

RULE INPUT	COMES FROM	DECIDES	WHY A PLUGIN CANNOT FORGE IT
Who — claims	Xano `user_claims`: identities keyed by each provider's immutable subject, returned by `/auth/me`	Which subject is acting, and through which login	Resolved on the server from a sealed token; never taken from a form field or an email match
What may be used — extras	Xano `user_extras`: profile and preference fields, cleared when consent is withdrawn	Which personal fields can be shown, sent or used	Consent state is a record with a time and policy version, not a checkbox the page sets
What is allowed — configuration	The tenant's configuration, managed in the Configuration System: which source owns which fields, each key's scope, each environment	Which actions, fields and rows a caller or import may touch	Changed only by its owners in the console; every change lands in the audit ledger
Proof — EdDSA signatures	Ed25519 keys held by the Worker on Cloudflare; public halves in the JWKS	That a mandate, entitlement or receipt was issued by the platform and not altered	The private key is never on the VM; anyone can check a signature on the verify plane

One request, in order:

1. **Cloudflare WAF and Access** filter the request at the edge.
2. **The Worker verifies the token's Ed25519 signature** against the published key id. A retired, unknown or altered token stops here.
3. `/auth/me` **returns claims and extras** for that subject — the server's answer, attached by the sidecar, never the browser's.

4. **The configuration's rules decide**: this subject, this tenant, these fields, this action. The default is deny; anything not written as allowed is refused, and the refusal is recorded.
5. **Xano's endpoint precondition** applies the same rule at the data layer and filters by tenant before Postgres runs the query.
6. **WordPress renders the result.** It never held a key, a password table or a rule.

"By design" means the rules exist before the first request, not after the first incident: the default is deny, every input comes from a system the theme cannot write, and every decision is signed and logged. Moving from a shared VM to one isolated VM is the infrastructure step; RuBAC is what makes the move hold when a plugin is compromised anyway.

Sources: Hostinger technology (CloudLinux LVE); WP Engine Global Edge Security; Cloudflare Tunnel. Plan features change; check your host's current documentation.

The safer customer option: let Shopify hold the customer and the card

For a store that wants to keep its WordPress or Webflow site, the most effective PII and PCI reduction is to stop holding the data at all. Shopify's Storefront Web Components put products, prices and a cart on any page with a few lines of HTML, and the **Buy now** button sends the shopper to checkout on Shopify. Shopify's own WordPress plugin — built by Shopify, not by WordPress — does the same from the WordPress Editor: products are managed in Shopify and placed on WordPress pages and posts.

The components are plain HTML elements plus one script, so they are not tied to a CMS. The same block works in a Webflow embed, a WordPress theme or

block, an Astro or EmDash component, an 11ty template, and an AEM component or Edge Delivery block — anything that renders HTML and can load a script. Try them in Shopify's playground.

A working example. crm-base.webflow.io is a Webflow site with no commerce code of its own. When a shopper adds a product to the cart, a CRM Sync embed mounts Shopify's `<shopify-store>` and `<shopify-cart>`, and checkout lands on the Shopify store at www.crm-sync.dev. The Webflow site never sees a card number or keeps a customer record; the Shopify store holds both.

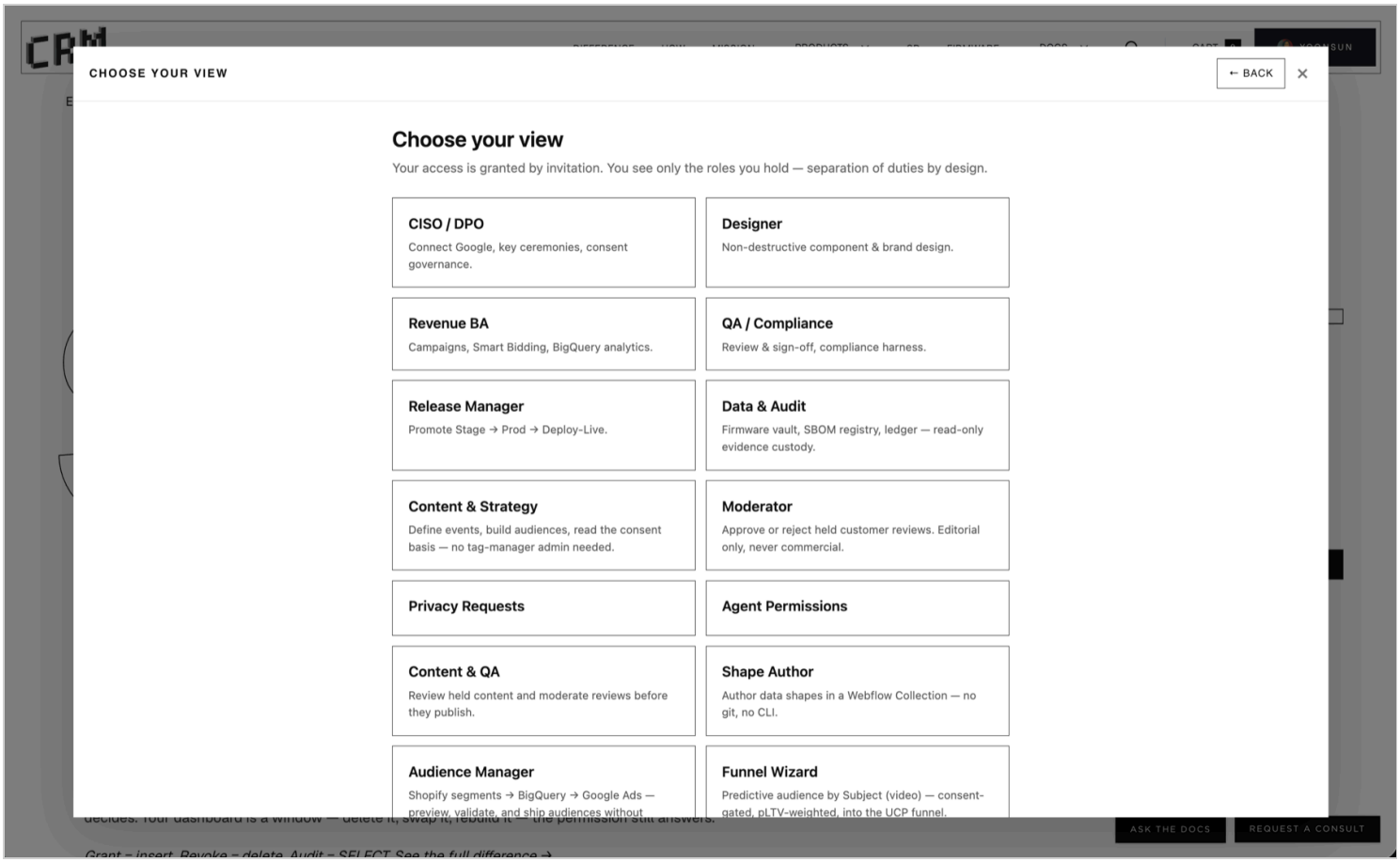
The same theme renders the same everywhere. The design is the Webflow export plus one embed block, with no framework code in it, so the same theme renders the same on WordPress, AEM, 11ty, Shopify, Next, Astro, Nuxt, Svelte and Drupal — with the forms and the data-binding security intact. The security does not live in the theme, so it does not change when the host does: forms still post to the Worker and hold a typed email until the inbox confirms, `/auth/me` still decides who the caller is, permissions are still granted rows checked on every request, and the cart and checkout are still Shopify's. None of those hosts ever holds a customer record, a consent record or a card.

The structured data carries over too. The JSON-LD for collections and their joins, the `llms.txt` answer engines read, and the LLM weighting are generated by the Worker from the records, not by the theme. Each AEO taxonomy item carries a weight — the summed predicted value of that channel's consented members, an aggregate and never a per-person score — and the JSON-LD and agent surfaces rank by it. Move the theme to another host and what an answer engine reads, and how it is ranked, stays the same.

Stakeholder teams review the model

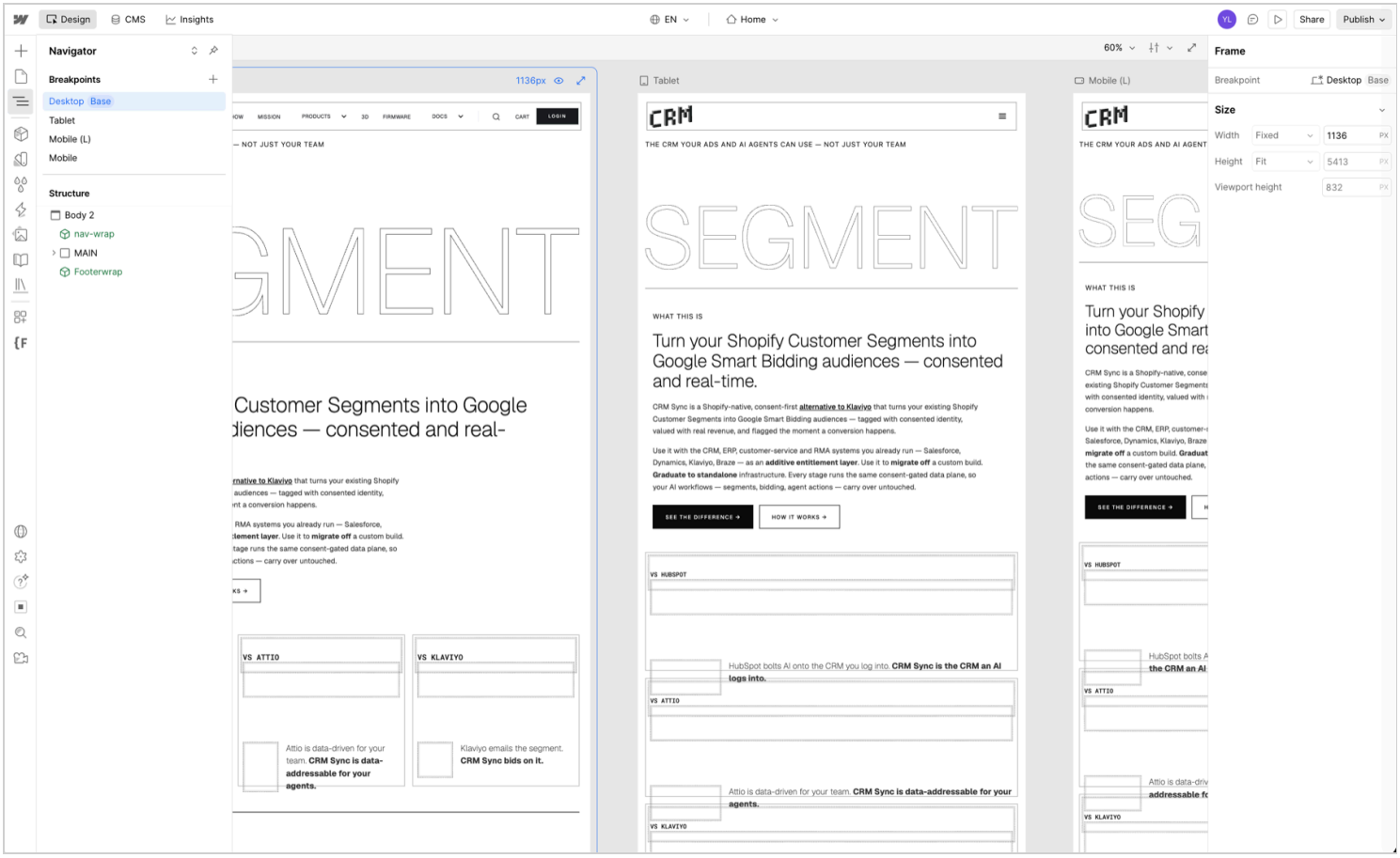
The modeling is intentional, and it is reviewable. Because the collections, joins, weights and permissions are records rather than theme code, the people accountable for them can read and adjust them without a developer

redeploying a site: subject-matter experts and QA review the **content**, developers own the **schema and joins** behind the release gate, and revenue stakeholders adjust the **context** — the taxonomy, channels and weights that decide what is ranked. **Mandate permissions** are adjusted the same way: an agent's mandate can be tested and revoked from the console, and the roles are separate by design — the role that tests a mandate before a release depends on it cannot revoke it. Each change is a recorded row, so every adjustment can be checked after the fact.



Signed in as the author on www.crm-sync.dev. The name in the navigation comes from `/auth/me` — the claims and extras for that subject, read on the server, not stored by the theme — and the views listed are exactly the roles that subject was granted. Access is by invitation; someone holding only Revenue BA sees campaigns, Smart Bidding and BigQuery analytics and nothing else. QA / Compliance reviews and signs off, Content & Strategy defines events and audiences, Agent Permissions holds the mandates, and Shape Author edits the

data shapes in a Webflow collection with no git and no CLI. The WordPress and Astro/EmDash paths are in File System Agnostic Publishing, and the AEM path in Shopify / Google Integration on an AEM Scaffold.



The crm-base design in Webflow at three breakpoints. The structure is a navigation component, the page content and a footer component — design only. The CART link in the navigation opens Shopify's cart; the products, cart and checkout come from Shopify, not from Webflow.

OPTION	WHAT THE SITE HOLDS	WHAT SHOPIFY HOLDS	WHY IT IS SAFER
Storefront Web Components on Webflow, WordPress, Astro/EmDash, 11ty or AEM	Markup, the store domain, and optionally a public, read-only Storefront token (needed only for inventory counts and custom data)	Catalog, prices, cart, checkout, customer accounts, payments	No customer table, no order table, no card flow and no secret key on the site; a breach of the site has no customer database to take
Shopify's WordPress plugin	WordPress pages and posts that embed Shopify products	Products and checkout; Shopify states PCI, SOC and GDPR compliance are built in	WooCommerce's customer and order tables never exist in the WordPress database
WooCommerce or Magento on the site's own server (for contrast)	Customers, orders, addresses, often payment-flow scripts	Nothing	Every row of PII sits in the same PHP application as every plugin

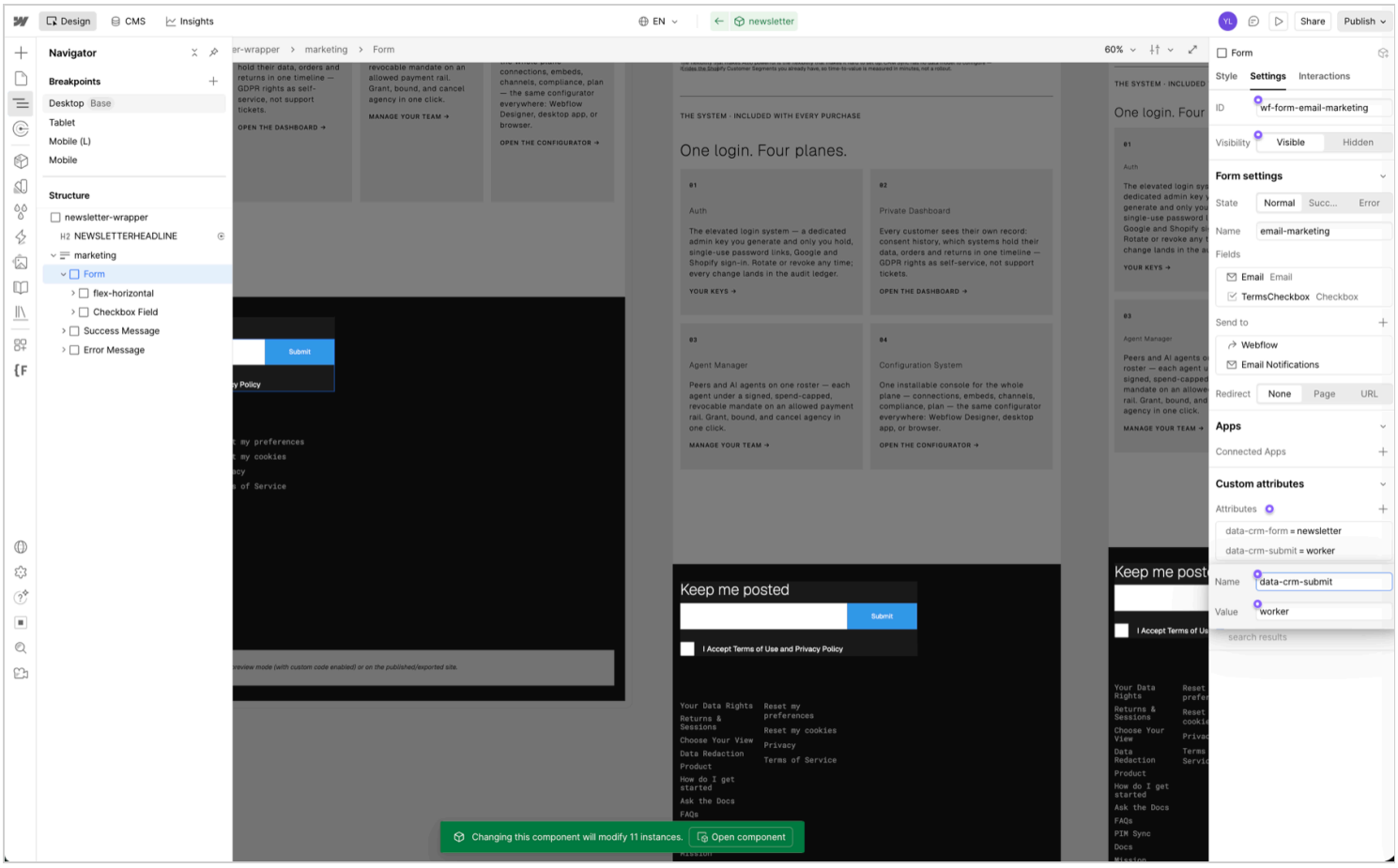
What it does not remove. The page that holds the Buy now button is still a page you serve, and the October 2026 hijack worked by changing what happened *before* checkout. The rules above still apply to it: no client-side redirects to other domains, a Content Security Policy that lists every script, an inventory of the scripts on any page that links to checkout, and a private-window test checkout on every release. WordPress still runs PHP for the content site, so the runtime patches still matter. Only the public Storefront token belongs in the page — never an Admin API token. Shopify's plugin page does not say where customer and order records live; confirm that before describing the plugin to customers as keeping them off WordPress.

Setup, including the Webflow-to-WordPress and EmDash/Astro paths, is in File System Agnostic Publishing → the Web Components block.

Secure forms on any front end

A form is the other place PII enters, and the platform's own form storage — Webflow submissions, a WordPress form plugin's table, a Liquid contact form — puts that PII next to everything else the platform runs. CRM Sync forms send the data to the Worker instead, and the form itself is only HTML:

```
<form data-crm-form="newsletter" data-crm-submit="worker">
  <input type="email" name="email" required>
  <label><input type="checkbox" required> I accept the Terms of Use and Privacy Policy</label>
  <button type="submit">Submit</button>
</form>
<script src="https://crm-sync.dev/embed/forms.js" defer></script>
```



The crm-base newsletter form in Webflow. The design stays in the Designer; the two custom attributes hand the submission to the Worker.

Because the contract is two attributes and one script, the same form works wherever HTML renders:

FRONT END	WHERE THE FORM LIVES
Webflow	A form element with the two custom attributes, as above
Astro / EmDash	A component that renders the `<form>`; the script in the layout
Next.js	A component that renders the `<form>`; the script through `next/script`
11ty	An include or shortcode; the script in the base layout
Shopify	A theme section or snippet in Liquid; the script in `theme.liquid`
WooCommerce / WordPress	A block or template part; the script enqueued by the theme
AEM	A component or Edge Delivery block that emits the `<form>`

What makes it secure.

- **The platform never stores it.** The script intercepts the submit before the platform's own handler runs and posts to the Worker, so the address is not written to Webflow, WordPress or Shopify form storage.
- **A typed email proves nothing.** Without a session, the Worker records nothing and mails a single-use, 48-hour confirmation link; the permission is written — with its date, time and the exact wording shown — only when the inbox answers. A signed-in shopper's session proves the address, so theirs is recorded immediately.
- **The reply never reveals an account.** Every unconfirmed submission gets the same answer, whether or not the address already has an account.
- **Abuse is capped.** 60 submissions an hour per IP, three confirmation mails a day per address.

The demo, on our own data. This is the live run from 22 October 2026, submitted from no session with the author's own address (masked here):

```
POST https://crm-sync.dev/forms/newsletter

{"email":"y••••••••••@gmail.com","page":"/verify-email-is-not-identity","locale":"en"}
```

```
{"ok":true,"form":"newsletter","form_id":"newsletter_en_2026-10-22T12:02:22.862Z_f767c744",

 "locale":"en","status":"confirmation_sent",

 "why":"check your inbox — the request is recorded once the address confirms it"}
```

No user ID, no "created" flag, and no consent row until the link in the inbox is clicked. The `form_id` is the join key: the same value is written to the reconcile log, the GA4 signal event and the response, and it survives confirmation, so the held entry and the recorded one join on it.

Email-only plugin forms and fee-based middleware. Most CRM and plugin forms capture an email address and nothing else — no consent wording, no time, no session — and store it in the vendor's own database, often a PHP application outside the merchant's control. Fee-based middleware then copies it onward, charging per task or per record. Moving that audience to Google is not a file transfer: Google's EEA consent rules require `ad_user_data` and `ad_personalization` consent for the people in it, and an address with no consent event behind it has nothing to show. Migrate with the consent, not just the address — export what consent evidence exists, send the same single-use confirmation to everyone whose record has none, and connect the confirmed audience to Google's APIs directly from the Worker, which also ends the per-record middleware fee.

What it does not cover. If the script is blocked — an ad blocker, a strict CSP that omits it — the platform's own handler takes the submit, and the address lands in the platform's form storage after all. Remove the platform's own destination

where the platform allows it, list `crm-sync.dev` in the CSP, and test the form with the script blocked.

Packaging the form. The HTML contract above is what ships today. Because the form is a contract — a slug, its fields, its consent wording — it can travel in other shapes:

SHAPE	WHAT IT CARRIES	STATUS
HTML + `forms.js`	The form and the submit interception	Built
Worker route `/forms/<type>`	Validation, the hold, the confirmation mail, the consent record	Built
`form_id` join key	One identifier across the reconcile log, GA4 event and consent record	Built
A Rust crate	The form's field schema and validation as a type, compiled to WebAssembly and checked inside the Worker — the same shape check used for AI task binding	Pattern, not built
Joinable data for 11ty, Liquid or Markdown	The form definition as data — `_data/forms.json` in 11ty, a Markdown front-matter block, or a Liquid snippet that reads it — so every front end renders the same fields and wording from one record	Pattern, not built
A Shopify metaobject	The same definition as a metaobject entry, so the theme renders the form from data the store already governs	Pattern, not built

Shopify plugins become Rust Shopify Functions

OLD PLUGIN PATTERN	REPLACE WITH	WHY
Line-item Script — tiers, BOGO, loyalty redemption	Discount Function	Runs inside Shopify on declared input; no merchant server to breach
Shipping Script	Delivery customization Function	Same
Payment Script	Payment customization Function	Same
Upsell or bundle app editing the cart	Cart transform Function + theme app extension	No third-party server holding order scopes
checkout.liquid fields and validation	Cart and checkout validation Function + Checkout UI extension	No arbitrary script on the payment page
Thank-you page script tag for points or pixels	HMAC-verified order webhook to a ledger Worker, plus a consent-aware web pixel	Events verified; tracking honours consent
PHP app server on the REST Admin API	GraphQL Admin API from a Worker	Meets the GraphQL-only rule; no PHP host

Shopify recommends Rust for public apps, large carts and complex logic, because it compiles directly to WebAssembly and stays inside the instruction-count limits; JavaScript is for prototypes. Functions decide; they do not store. Points, history and consent stay in the ledger, behind `/auth/me`.

Identity, data and AI tasks

1. **Identity.** A sidecar resolves every request through Xano `/auth/me` with the caller's JWE token and forwards the actor in a signed internal header. PHP never parses tokens and never trusts a `user_id` sent in a form.

2. **Data.** Postgres row-level security filters every read and write by tenant and actor, so a plugin bug or SQL injection returns only the caller's own rows. MySQL has no RLS; WooCommerce and Magento data either moves or sits behind filtered views.
3. **AI tasks.** Each automated job runs in a Tool Runner loop with a scoped tenant key and only the tools that task needs — the runner exists in PHP too. The full model — Cloudflare binds, Xano grants, Rust checks — is in Entitlement Security → AI task binding.

AI data binding with provisional permissions. A task binding says which tools an agent may call. The data it may touch is bound the same way — by permissions that expire, and by gates on where the subject is, who they share with and what device they used:

GATE	WHAT IT CHECKS	STATUS
Provisional access	Every grant carries a start and end date, and each permission can carry its own expiry; both are checked on every permission decision, so a time-limited grant ends without anyone revoking it	Built
Sovereign nation	A per-country policy, keyed by ISO code, read before an order exists: sovereign markets require a named permission (Korea's domestic settlement, for example), sanctioned or blocked markets are refused, and an unreachable policy fails closed	Built
Peer	Whether one subject may share with another	Partial — the check is written and tested but no route calls it yet; sharing is governed by single-use, expiring invitations today
Device	The platform, device type and browser behind each consent event	Recorded as provenance on every consent row; refusing access by device is not built

HOP	PERMISSIONS	TLS TRANSPORT
Browser or agent → edge	Access policy, or the Xano JWE	TLS 1.2 minimum; HSTS
Edge → origin	Origin accepts only Cloudflare	Tunnel; no inbound ports
Runner → tools	Scoped tenant key, per-task tool list	HTTPS; Access service tokens
Worker → Xano	Workspace-scoped key; permission check in Xano	HTTPS; a shared-secret header so the direct Xano URL refuses traffic that skipped the Worker
Shopify → store logic	Function input query; webhook HMAC, constant-time compare	Shopify-hosted; HTTPS webhooks
Service → database	Role per service; actor and tenant set per request	`sslmode=verify-full`

The plugin in the middle: data ownership from Shopify to CRM, service, fraud, consent and Google pLTV

A typical store sends one customer's data through several systems, each connected by a plugin:

- **Marketing and CRM:** Klaviyo, Salesforce
- **Customer service:** Gorgias, Sprinklr
- **Consent management (CMP):** OneTrust
- **Fraud:** Signifyd, Stripe Radar
- **Advertising and predicted lifetime value (pLTV):** Google Customer Match, enhanced conversions and BigQuery

The gap is the plugin in the middle. It holds the access tokens for both sides, and nothing it does leaves a record the merchant controls. The merchant is the controller under GDPR and CCPA, but the evidence of what was sent, with what consent, and whether it was erased sits with the processors.

Where ownership breaks:

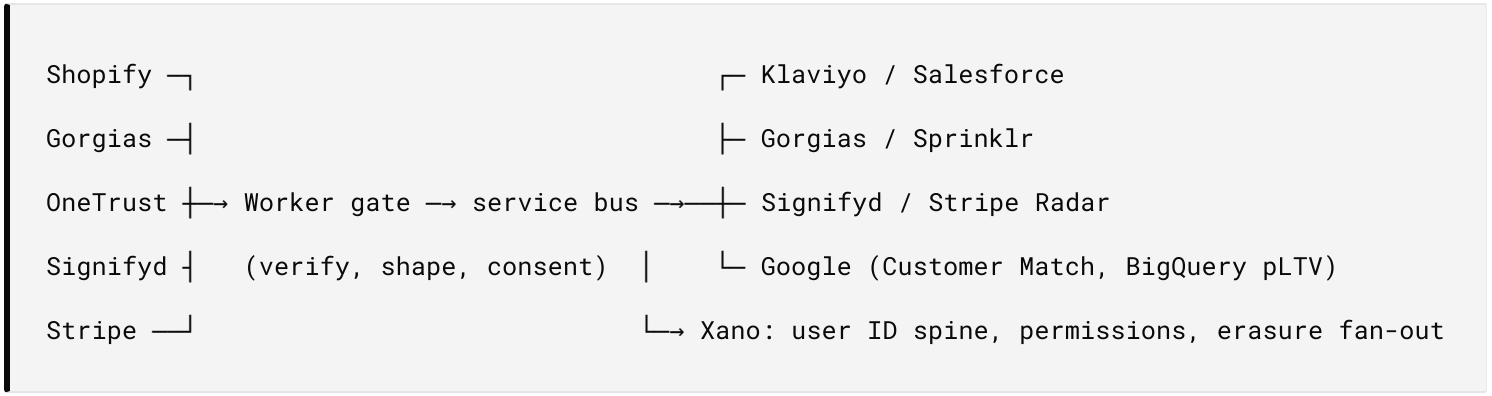
HOP	WHAT MOVES	WHERE OWNERSHIP BREAKS
Shopify → plugin	Customers, orders, marketing consent	The plugin's own credentials read the data. From here the merchant has no record of what was taken, only the vendor's logs
Inside the plugin	Field mapping, identity matching	The plugin's mapping table becomes the real identity record (Shopify ID ↔ Klaviyo profile ↔ Salesforce contact). The vendor owns it, and it disappears with the plugin. Unmapped fields are dropped silently, consent fields often among them
Plugin → Klaviyo or Salesforce	Profiles, events	Each system keeps its own consent fields. A two-way sync where the last write wins, or a nightly CSV or ERP import treated as the master record, can replace a customer's latest choice with an older value
Shopify → Gorgias or Sprinklr	Order history, addresses, message threads	The ticket becomes a second customer record, outside the store's permissions. Customers paste card numbers and ID documents into messages, bringing PCI and sensitive data into a system that wasn't scoped for it. Agents can often refund or edit orders from the helpdesk with the app's token, not their own
CMP (OneTrust) ↔ Shopify	Consent choices	The consent receipt lives in the CMP, the storefront signal lives in Shopify's Customer Privacy API, and each downstream system keeps its own copy. Nothing proves the three agree at the moment data was sent
Order → Signifyd or Stripe Radar	Order, device, IP, payment signals	The fraud decision is made and stored by the vendor. It rests on fraud prevention as its legal basis, not marketing consent, so it must never be reused for advertising. A plugin that copies risk scores into the CRM or an audience list crosses that line
Klaviyo or Salesforce → Google	Hashed emails, order values	Google requires `ad_user_data` and `ad_personalization` consent, but those signals were rarely carried from Shopify; they are usually inferred from a marketing opt-in. The pLTV model is then trained on data whose origin and consent nobody can prove
Erasure, back through the chain	Deletion requests	Shopify's `customers/redact` webhook reaches the app it was sent to, not the copies in the CRM, helpdesk, fraud vendor, BigQuery or Google audiences

Every hop is also a place code can be injected. Each connector runs third-party code where PII and card data change hands: a checkout script, a webhook receiver, an import job, a helpdesk widget. A distrusted certificate authority (the

emSign case) makes it easier to impersonate any of those endpoints, but the injection itself happens in whatever code runs at the handoff. That is why each hop needs a verified sender and a declared shape, not just TLS.

THE FIX: ONE SERVICE BUS WITH TIMESTAMP-VERIFIED RECORDS, LINKED TO THE USER ID

The plugins can keep doing the transport. They stop being the only place the evidence lives. Every transfer, in either direction, also writes one record to a **service bus** the merchant owns, keyed to the merchant's own user ID.



1. **One user ID spine the merchant owns.** Xano, behind Cloudflare, holds a pseudonymous ID for each customer. Every vendor's ID (Shopify customer, Klaviyo profile, Salesforce contact, Gorgias customer, OneTrust identifier, Stripe customer) is linked to it. The bus carries the spine ID, never the email address.
2. **Verify the sender at the gate.** Webhooks are checked by their HMAC or signature (Shopify, Stripe and Signifyd sign theirs; for any vendor that doesn't, use a per-vendor secret URL behind Cloudflare Access) before anything is read, then parsed into a declared shape by the Rust gate. Unknown fields and oversized bodies are refused.
3. **Consent travels with the record.** The gate reads the consent state at the moment of the transfer (CMP receipt plus Shopify signal) and writes it into the record. Google receives only what was actually granted, never a value inferred from something else.

4. **Purpose is part of the record.** Fraud and service records are tagged with their purpose (`fraud_prevention`, `support`) and can't be routed to marketing or advertising destinations. Card numbers and ID documents are redacted from service messages before the record is stored.
5. **Every record is timestamped, chained and signed.** Clocks differ between clouds, so a timestamp alone doesn't prove order. Each record carries:

```
{
  "seq": 18422,
  "prev": "sha256:9f2c…",
  "subject": "usr_7f3c…",
  "event": "fraud.decision",
  "source": "signifyd",
  "destination": "shopify.order",
  "purpose": "fraud_prevention",
  "consent": { "analytics_storage": "granted", "ad_user_data": "denied", "ad_personalization
  "observed_at": "2026-10-24T14:02:09Z",
  "received_at": "2026-10-24T14:02:11Z",
  "kid": "bus-2026-10",
  "sig": "ed25519:…"
}
```

   - `observed_at` is when the source says it happened; `received_at` is the gate's own clock. A large gap between them is flagged.
   - `seq` and `prev` (the hash of the previous record) fix the order and make a deleted or edited record visible, whichever cloud's clock was right.
   - `sig` is an Ed25519 signature from a key the merchant holds, so the record can be checked anywhere: in BigQuery, by an auditor, or after leaving a vendor.

    - Where a record may be needed as legal evidence, also anchor batches with a trusted timestamp (RFC 3161) from a time-stamping authority.
6. **Erasure fans out from the spine.** One request finds every linked vendor ID and sends each vendor its deletion call. Each confirmation is written back to the bus as its own signed record.
7. **Analytics only read.** BigQuery and the pLTV model read the bus to score customers. They never decide access; permission stays in Xano.

What the merchant can then answer, for any user ID, from records it owns: what was sent to which vendor and when; under which consent and purpose; which vendor decisions (fraud, refunds, service actions) changed the order; and whether every copy was erased.

Status. Built today: consent records with device provenance, Shopify webhook HMAC checks in constant time, consent and erasure events in a hash chain ordered by `seq` and `prev_hash` and signed with Ed25519, and erasure fan-out to Shopify, Klaviyo, Salesforce, Adobe, Google Customer Match, the pLTV score and any agent mandates, with the Google instruction written to the chain. Not built yet, and pinned as failing tests in the CRM Sync test harness: erasure to helpdesk, fraud and CMP vendors (Gorgias, Sprinklr, Signifyd, OneTrust), purpose tags that keep fraud and support data out of ad destinations, `received_at` with clock-skew flags, the Rust shape gate, and one table linking every vendor ID to the user spine.

Google Merchant API: rebuild loyalty, reviews and comparison in one pass

The Google feed integration is being rebuilt regardless. The Content API for Shopping shut down on 18 August 2026; Merchant API became generally available as v1 in July 2025, its v1beta versions were discontinued on February

28, 2026, new features ship only there, and conversion sources move from `content/v2.1` to `conversions/v1`. Connect loyalty, reviews and price comparison directly in the same conversion, rather than through a plugin that can be delisted.

GOOGLE SURFACE	MERCHANT API	NEEDS	REFERENCE
Loyalty — member prices, points, loyalty shipping	`loyaltyPrograms[]` on product attributes	A program linked in Merchant Center; AU, JP, UK, DE, FR, US; member prices matching feed, landing page and checkout	Product attributes · Attribute spec
Product reviews	Reviews sub-API, `productReviews`	Product ratings enrolment; `is_verified_purchase` set from a real order	Guide
Seller reviews	Reviews sub-API, `merchantReviews`	Store ratings enrolment	Guide
Price comparison	Reports sub-API — price competitiveness, price insights, competitive visibility	Standalone or individual sub-account	Reports
Comparison Shopping Services	CSS API; `Csses` folds into Accounts	A CSS partner	CSS
Conversions	Conversions sub-API, `conversionSources`	Field renames: `property`, `destination`, `report`	Migrate

Conversions at no ad spend. Merchant Center conversion sources report conversions from free listings — the organic surfaces where reviews, loyalty member prices and price comparisons appear — as well as from ads. They only match exactly when each event carries three things, captured at the moment it happens:

CAPTURED	WHY IT MAKES THE MATCH EXACT
The GA4 session ID	Ties the conversion to the visit that produced it, not to a guess about the visitor
Device provenance	Platform, device type and browser, recorded with the consent event, so the same person on two devices stays two provable sessions
An idempotency key	Derived from the subject or session, the event, its time and its source (`form_id`), so a retry or a double-fired tag is one conversion, never two

That is what turns an AI answer into an attributable sale: the assistant finds the candidate, and the session, device and key decide the transaction. CRM Sync writes all three today for a signed-in submission. A submission confirmed by email deliberately drops the typing browser's GA identifiers — carrying them would bind a stranger's browser to the person who clicked the link — so it keeps the device and the key but not the session. Conversions sent without all three are estimates.

A verified-purchase review or a member price shown to Google is a claim about a real order and a real member. CRM Sync records that claim as a signed mandate anyone can check offline in a browser — verify a mandate · verified transactions.

Supply chain: every new ecosystem is a new door

Moving off PHP brings in Python for AI gateways, npm for Workers and Rust crates for Shopify Functions. On March 24, 2026, LiteLLM 1.82.7 and 1.82.8 were published to PyPI with a credential stealer — one through a `.pth` file that runs on every Python start — taking SSH keys, cloud and Kubernetes credentials, `.env` files and TLS private keys (BleepingComputer). A second

poisoned release, 1.107.0, followed on October 21 — Entitlement Security → LiteLLM.

ECOSYSTEM	PERMISSIONS CHECK BEFORE INSTALL	WHAT IT CATCHES
Rust crates	`cargo vet --locked` in CI: every version covered by a trusted published audit, or reviewed and certified	A hijacked release nobody has reported yet
Python	Hash-pinned installs; versions at least two weeks old	A new malicious release
npm	Committed lockfile; `npm ci --ignore-scripts`; an osv-scanner baseline with expiry dates	Install-time scripts; known-bad versions
GitHub Actions	Pinned to commit SHAs	A retagged action

CRM Sync's desktop build imports the published crate audits of Mozilla, Google, the Bytecode Alliance, ISRG, Embark and Zcash — 102 crates fully audited, 357 recorded as a starting baseline — and fails CI on any crate version no audit covers. Wildcard trust is avoided on purpose: a hijacked publisher would pass it.

Signed records: SBOMs, firmware and signing keys

Every incident in this article ends with the same question from the board, the insurer and the regulator: *were we affected, and can you prove it?* A business that has to answer by meeting finds out in weeks. One that keeps signed records answers by query in minutes. Three records do the work:

- **A software bill of materials (SBOM)** is the ingredient list for each release: every package, runtime and certificate bundle, with its version. When a flaw is announced, the SBOM says which releases contain it.
- **A signed firmware or build image** is the sealed box. Its hash is certified when it is uploaded, so a distributor, a device or an auditor can check that what

they received is exactly what was built.

- **A signing key** is the seal itself. Its private half stays on the server. Its public half is published, so anyone can check a receipt, a mandate or a certificate without asking the issuer.

CRM Sync publishes all three. Anyone can check a mandate offline in a browser on the **verify plane**, against the public keys at `/.well-known/jwks.json`. The app's own SBOM is public, and firmware images go to the firmware vault. The vault gives each image its own encryption, an upload certificate, grant-gated downloads, and a hash-chained ledger of every grant, denial and download.

Business summary: the verify plane and the firmware vault

The need. After a breach, three groups ask for proof, not reassurance. Customers and partners want to know a receipt or permission is genuine. Distributors and device owners want to know a firmware image is the one that was built. Regulators and insurers want to know who had access while the flaw was open. Under the EU Cyber Resilience Act, those answers are due in 24 hours (early warning), 72 hours (notification) and 14 days (final report) from 11 September 2026. A business that answers from email threads and storage links misses those deadlines.

SERVICE	THE BUSINESS NEED	SECURITY BENEFIT
Verify plane	Partners, auditors and agents must trust a mandate, receipt or entitlement without a call to us	The check runs in the reader's own browser against the public key at `/.well-known/jwks.json`. No account, no API call and no trust in CRM Sync is needed. One changed character reads INVALID, and a token from a retired key is refused
Firmware vault	Firmware and build images are shared as storage links that, once sent, are shared forever, with no record of who fetched them	Each image gets its own AES-256-GCM key, wrapped under a versioned key and stored apart. Uploads get an Ed25519 certificate; downloads need a grant and a 120-second single-use token; every grant, denial and download is sealed into a hash-chained ledger
Both, together	One answer to "were we affected, and who had it?"	The SBOM names the affected releases, the vault's ledger names who downloaded them, and the verify plane lets anyone check the certificates. A suspected leak is fixed by one key rotation, with no re-upload and no recall

What it replaces: a signed installer behind a link, a support inbox for "is this genuine?", and an incident report assembled by hand. **What it does not replace:** patching. The records prove what happened; the fixes in the timeline below still have to be applied.

Firmware identity: the part-number gap

Signed records assume you can say what a record is *about*. For firmware, that is harder than it looks. A buyer orders by manufacturer part number (MPN). Firmware is tied to the device by technical identifiers instead, and **there is no standard link between the two.** The mapping usually lives in the manufacturer's internal systems.

Where each identifier lives today:

WHERE	WHAT IDENTIFIES THE DEVICE	DOES THE MPN APPEAR?
The firmware update itself	UEFI capsules carry a firmware class GUID, one per model or component. IETF SUIT manifests (IoT) carry a vendor ID and a class ID, ideally a UUID derived from vendor plus model. Windows Update and LVFS/fwupd match hardware IDs computed from SMBIOS fields (manufacturer, product name, family, SKU). Components match on bus IDs: PCI vendor/device/subsystem, USB VID/PID	No. The update says "I'm for class GUID X"; the device says "I'm class GUID X"
The device	SMBIOS tables, populated by the firmware: manufacturer, product name, SKU number	Nearly. SKU is the closest on-device equivalent, but it is free text and filled inconsistently
The platform certificate	The TCG Platform Certificate records the platform's manufacturer, model, version and serial, and each component's manufacturer, model, serial and revision	Model, not orderable MPN. It is the closest signed record of model identity to hardware configuration; firmware version appears only indirectly, as a component revision
Attestation reference values	In the IETF RATS model, a CoRIM/CoMID manifest — a signed list of known-good measurements — names a device class by vendor, model and boot layer and lists its expected firmware measurements. TCG Reference Integrity Manifests do the same for PCs. A verifier compares DICE or TPM measurements against them	No. This is where firmware hashes formally meet model identity, but "model" is the manufacturer's own string
SBOMs	CycloneDX and SPDX describe firmware by supplier, name, version, CPE or PURL; CycloneDX has device and firmware component types	Optional, and often skipped
Manufacturing and ERP	In the product's bill of materials, firmware is its own line item with its own part number and revision. A pre-programmed chip often gets a different part number from the blank one ("MCU MPN + firmware v2.3"); distributors that program chips do the same	**Yes — accurately, but privately**

The gap. The authoritative *MPN ↔ firmware version ↔ firmware hash* mapping sits inside the manufacturer's ERP and PLM systems. The public, cryptographic artifacts — capsules, SUIT manifests, CoRIM, platform certificates — speak in GUIDs and model strings that do not map cleanly to orderable part numbers.

A distributor, an integrator or a buyer validating devices has no signed way to answer: *this MPN, at this revision, should be running exactly these firmware hashes.*

That is a catalog and identity-resolution problem as much as a security one, and signed records close it:

- **A signed registry row** that binds MPN, revision, firmware version, full SHA-256 and the linked SBOM, verifiable offline against a published key. The firmware vault already signs product, version, digest and SBOM in each upload certificate; carrying the MPN and revision on that row is what closes the gap for a buyer.
- **An as-of read:** "which firmware should this MPN have been running on 1 March?" is a question about history, answered from the ledger.
- **Actor identity:** who certified the mapping — the manufacturer, a programming distributor, an integrator — is provenance.

A trusted third-party issuer, eMudhra-style, could plausibly publish such mappings. The eMudhra incident is also the caution: the issuer's own keys must come from a sound random source, and the mapping must stay verifiable when the issuer is distrusted — which is why records should be signed with keys you can retire and republish, not only chained to a public CA.

See it working: the Protected Firmware Asset Management and Download Service vaults each image under its own key, certifies every upload, grants every download, and records each event in a hash-chained ledger — with SBOMs linked per image and certificates that verify against a public JWKS. The requirements behind it are in the AI Trust Framework Requirements.

The breaches, on one timeline

DATE	WHAT BROKE	LAYER	WHAT A SIGNED RECORD ANSWERS
30 April 2025	A key ceremony at the Indian certificate authority **eMudhra** generated eight intermediate CA keys with too little randomness. Nobody noticed for about 18 months	Signing keys	Which of our certificates chain through those intermediates
24 March 2026	**LiteLLM** 1.82.7 and 1.82.8 on PyPI carried a credential stealer, taking SSH keys, cloud credentials, `.env` files and TLS private keys	Python / AI gateway	Which builds installed those versions, and which keys that environment could read, so you know what to rotate
10–20 October 2026	The checkout hijack through trusted Shopify apps, with vendor-side configuration changed (above)	App identity	Which grants each app held, and when each one was used
17–20 October 2026	A researcher recovered all eight **eMudhra** private keys from the public certificates in under a second: six TLS intermediates and two code-signing intermediates. Chrome, Mozilla, Apple and Microsoft blocked them between 17 and 20 October. Chrome removes the emSign ECC Root CA - G3 from devices from 27 October	TLS / code signing	Which runtimes and trust bundles in our releases still accept the chain
20 October 2026	Go, NSS, Ubuntu, Debian and **Composer** — the PHP dependency manager that Drupal, Magento and Joomla projects are installed with — shipped fixes to their trust stores	PHP dependency chain	Which PHP estates have taken the updated certificate bundle
21 October 2026	**php2shell** (CVE-2026-109618 and CVE-2026-109619): unauthenticated remote code execution in the PHP runtime. This covers every WordPress, Drupal, Magento, Joomla and Laravel host, including its plugins, whatever the application code	PHP runtime	Which origins run an affected PHP build, by version, without logging into each server

DATE	WHAT BROKE	LAYER	WHAT A SIGNED RECORD ANSWERS
21 October 2026	**Node.js** CVE-2026-111497, a Brotli out-of-bounds write that allows remote code execution. The same releases — 22.23.4, 24.22.1 and 26.12.1 — also reject the eMudhra chains	Node runtime	Which containers, CI runners and self-hosted apps pin an older Node
21 October 2026	**LiteLLM** 1.107.0 on PyPI and Docker, the project's second compromise this year, again stealing signing keys and AI provider keys	Python / AI gateway	Which images resolved the floating tags, and which mandates were signed by a key that environment held

Details and sources are in Entitlement Security and in reporting on eMudhra by The Hacker News and Help Net Security.

What eMudhra teaches about signing keys

Who eMudhra is. eMudhra is an Indian digital-trust company. Its main lines of business:

- **India's largest licensed certifying authority.** Under India's IT Act it issues Digital Signature Certificates, which people and companies use to sign official filings: income tax, GST returns, company filings and government e-tenders. This is the core of the business.
- **eSign.** An online signing service built on Aadhaar, India's national digital identity.
- **emSign, its publicly trusted CA brand.** Its roots sit in the Chrome, Mozilla, Apple and Microsoft trust stores, so it can issue TLS and code-signing certificates that browsers accept worldwide. This is the part the incident hit, and one of the few Indian-owned roots in those programs. What failed was emSign's key generation, not its software or its algorithm: the 30 April 2025 ceremony for eight intermediates under emSign ECC Root CA - G3 produced computable keys. The emSign root key itself was not reported as weak.

- **Trust software for others.** Certificate-authority and authentication products for governments, banks and enterprises, in India and abroad.
- **A listed company,** public in India since 2022 and expanding into the Middle East, Africa, Southeast Asia and the US.

eMudhra says TLS is about 1% of its revenue, and its shares fell about 20% after the disclosure. Both numbers make the same point. A small, rarely run ceremony got little scrutiny, yet the failure hit the company's whole product, because what it sells is trust. India's Digital Signature Certificates run under India's own CA licensing rather than the browser root programs, so as far as has been reported they are not directly affected.

The keys were not stolen. They were computable. The ceremony used certified hardware security modules and still produced weak keys. A key is only as strong as the randomness it was made from, and an audited process is not a substitute for checking the output. Four properties keep a signing key's failure small:

PROPERTY	WHAT CRM SYNC DOES	WHAT IT LIMITS
Generated by the platform's cryptographic random source, never by hand or by a copied script	Mandate and entitlement keys are Ed25519, generated with WebCrypto inside the Worker	Weak or reused randomness
Public half published with a key id	Every token names its `kid`, and the verify plane rejects any `kid` that is not in the published JWKS	Trusting a key nobody can look up
Rotation with an end date	Keys rotate through a recorded key ceremony. A retired key keeps verifying what it already signed until its `not_after`, then stops	The useful life of a leaked key, without voiding honest receipts
Revocation you control	Retiring a CRM Sync key means editing our own JWKS. A browser vendor does not need to act	The days a public CA needs

Two of the eight distrusted intermediates issued code-signing certificates. Software and firmware signed under them lost their proof of origin with the chain. A vendor has to re-sign, and every customer has to trust the new signature. Images held in the firmware vault also carry their own upload certificate: the image's full SHA-256, its version and its SBOM, signed with an Ed25519 key that CRM Sync publishes. That certificate does not depend on any public CA, so a CA incident does not void it. Anyone can still check an image on the verify plane. It is a second, independent record, not a replacement for the vendor's own code signing.

LiteLLM is the reason signing keys never sit in an application environment. Both poisoned releases searched for signing keys and `.env` files. A key held as a platform secret and used only by the signing service is not on the disk the payload reads.

Signing keys you control: Xano encryption, AI binding and Cloudflare EdDSA

Every trusted CA can vouch for every domain. Certificate issuance is concentrated. In July 2024 W3Techs counted Let's Encrypt at about 64% of certificates on the top websites, GlobalSign about 24%, Sectigo 6%, GoDaddy 4% and DigiCert 2% (W3Techs, via Wikipedia). eMudhra is not in that top five, yet browsers trusted its intermediates exactly as much as the largest CA. A trust store is flat: the smallest root in it can issue a certificate for your domain. You can't choose which CAs other people's software trusts, so the keys that matter most should not depend on a public CA at all.

The stack splits the work so that a public CA secures only the connection, and everything that proves identity, authority or origin uses keys you hold:

LAYER	KEY	WHO CONTROLS IT	WHAT IT PROTECTS	EXPOSURE TO A CA FAILURE
TLS certificate	Issued by a public CA (Let's Encrypt for Xano, Google Trust Services for `crm-sync.dev`)	The CA	The connection, browser to edge and edge to Xano	Direct: a weak CA anywhere could impersonate the host
Xano encryption	`security.encrypt` / `security.decrypt` (AES-256) for fields at rest; `security.jwe_encode` / `jwe_decode` (A256KW key wrap, A256GCM content) for the sealed session	You, as Xano environment variables	Customer fields in Postgres, and the session token's contents	None: symmetric keys, no certificate and no CA in the chain
Cloudflare EdDSA	Ed25519, generated with WebCrypto inside the Worker and held as a Worker secret	You, through the key ceremony	Entitlements, mandates, firmware certificates, receipts	None: the public half is published in your own JWKS with a `kid`
AI binding	The same Ed25519 mandate key	You. The agent never holds it	What an agent may do, on which tenant, up to what amount, until when	None: the agent presents a signed mandate, and every call checks it against your JWKS

Why the work is split this way. Xano's security library generates RSA keys and elliptic curve keys on the NIST curves (P-256, P-384, P-521), and its JWS example signs with HS256, a shared secret. Ed25519 signing therefore stays in the Cloudflare Worker, and Xano does what it does best:

- encrypting rows and fields
- sealing the session as a JWE
- applying row-level rules before data leaves the database

Each side holds keys the other never sees:

- A leaked Xano encryption key does not let anyone sign a mandate.

- A leaked Worker key does not decrypt a customer row.

What this means after eMudhra:

1. **Fix trust first:** update pip, certifi, uv, Node and base images, as in Python and Docker after eMudhra. This repairs the one layer that depends on a CA.
2. **Narrow which CAs may issue for your domains.** A DNS CAA record names the CAs allowed to issue for a domain, and Certificate Transparency monitoring alerts you to any certificate issued anyway. Neither stops a browser from trusting a bad CA, but together they shrink and expose the window. Instructions are in CAA records.
3. **Leave your own keys alone unless they were exposed.** Xano encryption keys and Worker Ed25519 keys were never issued under emSign, so eMudhra gives no reason to rotate them. Rotate on their own schedule, or after an exposure like LiteLLM, through the key ceremony.
4. **Keep agents on mandates, not keys.** An agent that holds only a signed mandate has nothing a CA failure or a stolen environment can turn into new authority. See AI binding.

Sources: Xano security functions; W3Techs SSL certificate authority usage.

Layered encryption with Rust and WebAssembly: security additions shaped for AI callers

Why AI callers change the shape of the problem. An agent's requests are well formed, fast and plentiful. A prompt injection doesn't arrive as obviously malformed input. It arrives as a valid-looking tool call with an attacker's arguments: a different order id, a larger refund, a field the form never had. No single control catches all of that, so each layer below answers one question and refuses on its own. None of them tries to judge intent.

Encryption answers who can read a value, a signature answers who wrote it and whether it changed, and the Rust shape check answers whether this request is

one the system ever declared. Encoding (Base64) answers none of these: it only gets bytes through a text channel.

#	LAYER	WHAT IT DOES FOR AN AI CALLER	WHERE IT RUNS	STATUS
1	**TLS on every hop**	Nobody between the agent and the data can read or alter the request	Browser or agent → Cloudflare → Worker → Xano → Postgres	Built
2	**Sealed session (JWE)**	The agent carries an identity it can't read or edit: A256KW key wrap, A256GCM content	Issued by Xano, opened only by the data plane	Built
3	**Signed mandate (Ed25519)**	Who may act, on what, up to what amount and until when, checked against the published JWKS on every call	Cloudflare Worker	Built
4	**Rust shape check, compiled to WebAssembly**	The request must parse into a declared type before anything reads it: a size cap, no unknown fields, bytes kept apart from text, bad Base64 refused. A call the system never declared can't be constructed	Inside the Worker, in the Wasm sandbox	Pattern, not built
5	**Field encryption**	Sensitive fields are stored as ciphertext, so an agent or a leaked export that reads the row without a decrypt grant sees only scrambled bytes	WebCrypto AES-GCM in the Worker, or Xano `security.encrypt`	Available in both, applied per field
6	**Row rules (RuBAC)**	Even a valid, decrypted request returns only the caller's own tenant and records	Xano, before Postgres is touched	Built for tenant and entitlement checks
7	**Signed receipt**	Every action leaves a record a third party can verify without trusting the operator	Worker signs, ledger stores	Built for mandates and firmware vault certificates

What the Rust layer looks like. The allowed calls are a closed list. Serde refuses any tool or field that isn't on it, and the mandate's limits are checked against typed values rather than strings:

```
#[derive(serde::Deserialize)]
#[serde(tag = "tool", deny_unknown_fields)]
enum AgentCall {
    ReadOrder   { order_id: u64 },
    RefundOrder { order_id: u64, amount_cents: u32 },
}

fn admit(body: &[u8], mandate: &Mandate) -> Result<AgentCall, Refusal> {
    if body.len() > 16 * 1024 { return Err(Refusal::TooLarge); }
    let call: AgentCall = serde_json::from_slice(body).map_err(|_| Refusal::Shape)?;
    match &call {
        AgentCall::RefundOrder { amount_cents, .. } if *amount_cents > mandate.max_refund_cents
            Err(Refusal::OverMandate),
        _ => Ok(call),
    }
}
```

The call is refused rather than repaired. A prompt-injected "refund 500 dollars" against a 50-dollar mandate fails at the type and limit check, before any database or model sees it.

Why Rust and WebAssembly for this layer:

- **The enum is the allow-list.** A tool that isn't declared can't be represented, so there is no "unknown action" branch to forget.
- **Bytes and text stay apart.** Ciphertext and keys are `Vec<u8>`, and text is a `String` that must be valid UTF-8. Ciphertext can't slip into a text field, and a key can't land in a log as if it were a message.
- **The Wasm sandbox has no filesystem and no network.** The check sees only the bytes it is handed, so even a bug in it can't reach a secret.

- **One check, many surfaces.** The same crate compiles for the Worker, Shopify Functions, the desktop build and the browser. Every front end applies the same rule, not a copy of it.
- **Dependencies are reviewed.** `cargo vet` stops an unaudited crate version, which is the LiteLLM-style poisoned release, before it ships.

What it doesn't do. Rust prevents memory errors and undeclared shapes, not wrong business rules. A badly chosen mandate limit is still enforced exactly as written. Wasm also adds bundle size and start-up time to the Worker, so keep the checking crate small.

One encryption detail to get right. The example in Xano's documentation uses AES-256-CBC. CBC hides a value but doesn't detect tampering. Where a stored value must not be altered, use an authenticated mode such as AES-GCM, which WebCrypto in the Worker provides, or sign the value as well. Either way, never reuse a nonce or IV with the same key, and keep keys in Xano environment variables or Worker secrets, never beside the data.

These layers sit on the hops in Identity, data and AI tasks, and the binding model behind layers 3 and 4 is in Entitlement Security → AI task binding.

Use-after-free in business terms: where Rust, garbage collection, Cloudflare, Xano and TLS each fit

The analogy. A hotel checks a guest out but doesn't cancel their key card. The room is cleaned and given to a new guest. The old card still opens the door, and the old guest walks in on the new guest's belongings, or leaves something behind for them.

In software, the room is a piece of memory and the key card is a pointer to it. The program releases the room, keeps the card, and a later step uses the card anyway. An attacker who knows this can arrange for their own data to be in the room when the old card is used, and the program ends up doing what the attacker wants.

Why it matters to a business. It is one of the most common ways browsers and older server software get taken over. Many emergency browser updates and a long line of PHP security patches come from it. It is not a configuration mistake a team made. It is built into code written in older languages (C, C++ and the PHP engine itself), and the only fixes are to patch it or move away from it.

Who handles it at each layer. Each part of the stack does a different job, and none stands in for another.

LAYER	WHAT IT DOES	IN HOTEL TERMS	DOES IT STOP USE-AFTER-FREE?
Rust (the gate in front of PHP, on Cloudflare)	Won't build a program that uses a released room	No key card can be made for a room that has been checked out	**Yes**, by design, before the code ever runs
Garbage collection (JavaScript on a Webflow page)	Releases a room only when nobody holds a card for it	The front desk never re-rents a room while any card still opens it	**Yes, for page scripts.** A site's own JavaScript can't create one
Webflow loading (which scripts the page runs, and when)	Limits how many outside scripts get in, and keeps page sections tidy when they are swapped	Controls who gets into the lobby	**No.** It reduces how many visitors could try an old card on the browser's own rooms. It also prevents a harmless lookalike: buttons that stop working after a section reloads
The browser (Chrome, Safari)	Its engine is C++; its makers add protections and ship updates	The hotel chain's own maintenance crew	**Partly.** That is what emergency updates are for. Your part is keeping browsers updated
Cloudflare (where the Rust gate and the page load run)	Runs the Rust gate between the internet and the PHP store, and serves the page	The security desk at the entrance, built to the Rust standard	It brings Rust's protection to a PHP store without rewriting the store
TLS	Encrypts the connection	An armoured car between buildings	**No.** It protects data on the way, not the program receiving it
Xano permissions (claims, roles, row checks)	Decides who may see or change which records	Whose name is on the booking, and which rooms they may enter	**No.** It is about who is allowed, not how memory is handled. It limits the damage if something upstream is broken into

The decision it leads to.

- **If you keep WordPress, WooCommerce or Magento:** the PHP engine underneath is C, so use-after-free bugs will keep appearing in it. A Rust gate in front, on Cloudflare, turns most hostile requests away before they reach

PHP, using code that can't have this bug. Xano permissions then limit what any request that gets through can see. The next section sets this up.

- **On the storefront (Webflow, or theme parcels):** page scripts are already safe from it, because JavaScript manages its own memory. What you control is how much outside code runs: fewer embeds, pinned scripts with Subresource Integrity, and a strict Content Security Policy. Keep browsers updated.
- **What to tell a board:** this is a defect in old code, not something to configure away. Shrink how much of that code faces the internet: Rust at the entrance, garbage-collected code on the page, and permission checks on every record. Most of these patches then become routine maintenance instead of an emergency.

The short definition is in the Vocabulary under **Use-after-free (UAF)**.

If you have to keep Magento, WooCommerce or WordPress: reinforce in place with Rust and parcels

Not every team can migrate this year. Contracts, extensions, staff skills and budget can keep a PHP store where it is. The store can still reach AI-grade security without a rewrite. You don't make PHP safe for agents; you stop agents and untrusted input from reaching PHP directly, and you move the parts that matter out of it one module at a time.

The shape: PHP stays the origin for what it already does well, such as the catalogue pages, the theme and existing extensions. Everything around it is reinforced:

```
Shopper or agent
  → Cloudflare: WAF, Access, Turnstile
  → Worker: Rust shape gate (Wasm), mandate check, rate limits
      ├─ unchanged routes → PHP origin over a Cloudflare Tunnel (no open inbound port)
      └─ parcel routes    → Xano: identity, rows, entitlements, encryption
  ← the PHP theme renders the page; parcels inside it fetch live data from the Worker
```

LAYER 1. PUT A RUST GATE IN FRONT OF THE PHP ROUTES AGENTS AND ATTACKERS USE

The gate is a Worker route with a Rust crate compiled to WebAssembly. It parses each request into the shapes the store declares, under a size cap, and refuses everything else before PHP parses a byte. It doesn't change the PHP code.

PLATFORM	ROUTES TO GATE	WHAT THE GATE ENFORCES
WordPress	`/wp-json/*`, `admin-ajax.php`, `xmlrpc.php`, uploads	Declared routes and bodies only; `xmlrpc.php` refused; `/wp-admin` and `wp-login.php` behind Cloudflare Access; uploads re-encoded by the Rust stage before WordPress sees the file
WooCommerce	`/wp-json/wc/*`, `?wc-ajax=`, checkout, webhooks	Typed cart and order bodies; quantity and price fields as numbers within limits; webhook signatures checked at the edge; consumer keys never reach a browser or an agent
Magento	`/rest/V1/*`, `/graphql`, `/admin`	Declared REST paths and bodies; GraphQL depth and size limits; admin path behind Access; integration tokens held by the Worker, not by the agent

An agent never calls PHP with a store credential. It calls the Worker with a signed mandate, the gate admits a declared call, and the Worker makes the PHP or Xano call on its behalf. That is the layered model above applied to an estate you keep.

LAYER 2. REPLACE PLUGINS WITH PARCELS IN THE THEME

A **parcel** is one self-contained module dropped into existing theme code. It has four parts, one per layer of the stack, and none of them lives inside PHP:

PART	WHAT IT IS	OWNED BY
Markup	A small HTML fragment with a mount point, placed in the theme	The theme: a WordPress template part or block, a WooCommerce hook, a Magento `.phtml` block, or a Liquid snippet
Behavior	petite-vue for client-side reactivity, or HTMX for server-swapped HTML fragments	One script, hosted on the Worker with Subresource Integrity, loaded only on pages that use it
Look	The classes and variables designed in Webflow	Webflow, exported as design tokens and CSS, so the parcel matches the site without theme CSS edits
Data and permission	Live values and writes through the Worker to Xano	Xano holds the rows and the rules; the Worker enforces permissions. The UI only hides what a caller can't use

PHP renders a placeholder and nothing else. It never holds the API key, never stores the data and never decides who may see it. A vulnerable plugin can't leak what PHP never had.

petite-vue or HTMX:

- **petite-vue** (about 6 KB) for values that react in the browser: a points balance, a live price or stock count, a configurator, consent preferences.
- **HTMX** (about 14 KB) for lists and forms the server renders: order history, reviews, a support form. The Worker returns an HTML fragment and HTMX swaps it in.

One parcel, three PHP themes. The same loyalty balance in WordPress, Magento and on a Webflow page:

```
<?php // WordPress or WooCommerce theme: template part, or a hook such as woocommerce_after_add
<div class="crm-parcel points" data-parcel="points" v-scope="Points()" @vue:mounted="load()">
  <span class="points__value">{{ balance }}</span> points
</div>
```

```
<?php // Magento: app/design/frontend/<Vendor>/<theme>/Magento_Catalog/templates/product/points
<div class="crm-parcel points" data-parcel="points" v-scope="Points()" @vue:mounted="load()">
  <span class="points__value">{{ balance }}</span> points
</div>
```

```
<!-- The same markup in a Webflow embed, plus one shared behavior script -->
<script type="module">
  // a self-hosted, pinned copy of petite-vue on your Worker, loaded with Subresource Integrity
  import { createApp } from "https://your-worker.example/vendor/petite-vue.es.js";
  createApp({
    Points: () => ({
      balance: "—",
      async load() {
        const r = await fetch("https://your-worker.example/parcels/points", { credentials: "inc
        this.balance = r.ok ? (await r.json()).balance : "—";
      },
    }),
  }).mount();
</script>
```

An HTMX parcel needs no script of its own beyond HTMX. The Worker returns the rendered list, and only for the signed-in customer:

```
<section class="crm-parcel orders" hx-get="https://your-worker.example/parcels/orders"
         hx-trigger="load" hx-swap="innerHTML">Loading your orders…</section>
```

Parcels that replace common plugins:

PARCEL	BEHAVIOR	REPLACES	DATA
Points balance and member price	petite-vue	A loyalty plugin storing points in `wp_usermeta` or a Magento extension table	Xano ledger via the Worker
Live price and stock	petite-vue	Cached prices that go stale, and an overselling risk	The system that owns the value, read live
Reviews list and review form	HTMX	A reviews plugin with its own tables and admin	Xano, with verified-purchase checks
Secure form	HTMX, or the existing `forms.js`	Contact and lead-form plugins that store submissions in MySQL	Worker route `/forms/<type>` with the consent record (built)
Consent preferences	petite-vue	A cookie plugin that keeps consent only in the browser	Xano consent rows, one subject across every site
Order history	HTMX	Theme templates that query orders in PHP	Read through the Worker, scoped to the caller

TWO SECURITY DETAILS FOR PARCELS

- **Content Security Policy.** petite-vue evaluates its template expressions with `new Function`, so it needs `'unsafe-eval'` in the page's CSP. Keep its expressions to bound data, never to text a user wrote. Where a page must run under a strict CSP, use HTMX with `htmx.config.allowEval = false` and no `hx-on` handlers, or put the parcel in a Web Component with its own policy.
- **Credentials and origin.** A parcel calls the Worker with the customer's session cookie, set `HttpOnly`, `Secure` and `SameSite`. The Worker allows only the store's

own origins in CORS. Keys stay in Worker secrets and Xano environment variables, never in the theme, and never in `wp_options` or `core_config_data`.

ROLLOUT ORDER

1. **Gate first.** Put the Worker and Rust gate in front of the API routes, and move admin paths behind Access. Nothing in PHP changes.
2. **Close the origin.** Serve PHP through a Cloudflare Tunnel so nothing can bypass the WAF and the gate.
3. **Parcel the plugin with the most data.** That's usually loyalty, reviews or forms. Ship it as a parcel, turn off the plugin, and export, check and delete its tables.
4. **Repeat, one plugin at a time.** Each parcel removes a PHP attack surface and a store of customer data.
5. **Decide later.** Once the data lives in Xano and the behavior lives in parcels, PHP renders pages and little else. Replacing it then is a theme change, not a data migration.

Status: the `forms.js` parcel, the `/forms/<type>` Worker route and the `<crm-*>` Web Components are built. The Rust gate crate, a shared parcel loader and the parcels other than forms are a pattern, not built.

CAA records: name the CAs that may issue for your domain

What a CAA record is. A CAA (Certification Authority Authorization) record is a DNS record that lists the certificate authorities allowed to issue certificates for your domain. Every publicly trusted CA must check it before issuing. With no CAA record, any CA in any trust store may issue for you. That includes emSign, the eMudhra brand whose eight intermediate keys were exposed. A CAA record that doesn't list emSign (`emsign.com`) tells every CA that follows the rules not to issue for you under it.

What it does and doesn't do:

- **It does** stop a careful CA from issuing for your domain when the CA isn't on your list.
- **It doesn't** stop a browser trusting a certificate that was issued anyway. A CA whose keys are computable, as eMudhra's were, can skip the check. Certificate Transparency monitoring covers that gap: every trusted certificate is logged publicly, so an unexpected one shows up.
- **It doesn't** touch email. CAA governs certificates only, so MX, SPF, DKIM and DMARC are unaffected.

One record at the apex covers every subdomain unless a subdomain has CAA records of its own. That includes subdomains pointed at other platforms, such as Shopify, so list every CA that any of your hosts uses before you add the first record. Otherwise the next renewal fails and the site goes down.

STEP 1. FIND OUT WHICH CAS ISSUE FOR YOU TODAY

You don't need anyone's access for this, because certificates are public:

```
# Every certificate issued for the domain and its subdomains, from the public CT logs
open "https://crt.sh/?q=%25.example.com"

# The certificate each host is serving right now
for h in example.com www.example.com account.example.com; do
  printf '%-24s ' "$h"
  echo | openssl s_client -connect "$h:443" -servername "$h" 2>/dev/null \
    | openssl x509 -noout -issuer
done

# Any CAA records already published
dig +short CAA example.com
```

On `crm-sync.dev` this showed three hosts and two CAs:

HOST	SERVED BY	CERTIFICATE FROM
`crm-sync.dev`	Cloudflare	Google Trust Services
`www.crm-sync.dev`	Shopify, through Cloudflare	Google Trust Services
`account.crm-sync.dev`	Shopify customer accounts (DNS only)	Let's Encrypt, requested by Shopify

Hosts you don't run yourself, such as Shopify customer accounts or a SaaS custom domain, are the ones people forget. Leaving out their CA breaks the next renewal.

STEP 2. CHOOSE THE RECORDS

Use the CA's CAA identifier, not its brand name:

CA	VALUE
Google Trust Services	`pki.goog`
Let's Encrypt	`letsencrypt.org`
SSL.com	`ssl.com`
Sectigo	`sectigo.com` (older records use `comodoca.com`)
DigiCert	`digicert.com`

Add each CA twice: `issue` for ordinary names, and `issuewild` for wildcards such as `*.example.com`. Once any `issuewild` record exists, a CA that isn't listed there can't issue wildcards even if it's listed under `issue`. For `crm-sync.dev` the records were:

NAME	FLAGS	TAG	VALUE
`@`	0	`issue`	`pki.goog`
`@`	0	`issue`	`letsencrypt.org`
`@`	0	`issue`	`ssl.com`
`@`	0	`issuewild`	`pki.goog`
`@`	0	`issuewild`	`letsencrypt.org`

SSL.com is included because Cloudflare may fall back to it when renewing a proxied host.

On Cloudflare, once the zone has any CAA record, Cloudflare automatically publishes CAA records for every CA its Universal SSL uses. They don't appear in the dashboard list, but they do appear in DNS. After the records above, `crm-sync.dev` also published `comodoca.com` (Sectigo) and `digicert.com`. That's expected, and it's what keeps Cloudflare's own renewals working.

Optional: an `iodef` record (`0 iodef "mailto:security@example.com"`) asks CAs to report refused requests to that address. Not every CA does.

STEP 3A. ADD THEM IN THE CLOUDFLARE DASHBOARD

1. Open **DNS → Records** for the domain.
2. Click **+ Add record**.
3. Set **Type** to **CAA** and **Name** to `@`.
4. Set **Flags** to `0`.
5. Set **Tag**:
   - **Only allow specific hostnames** for `issue`
   - **Only allow wildcards** for `issuewild`
6. Enter the CA domain name, for example `letsencrypt.org`.

7. Click **Save**. Repeat for each row.

Add the CAs your hosts are using right now first. A gap of a few minutes doesn't matter, but if the first record names only one CA, every other CA is refused until the rest are in.

STEP 3B. OR ADD THEM WITH A SCOPED TOKEN

The key-ceremony rule applies: the person makes the credential, and it never enters an AI session or a file.

1. **Create the token:** My Profile → API Tokens → **Create Token** → Custom token.
   - **Permissions:** Zone → DNS → Edit.
   - **Zone Resources:** Include → Specific zone → your domain.
   - **Expiration:** 7 days.
2. **Check the summary before creating it.** It must read `<your domain>, DNS: Edit` with an end date. If it says **All zones across all accounts** or **No expiration**, go back and fix it. A token like that can rewrite the DNS of every domain you own, for good.
3. **Run this in your own terminal.** The token is read without being shown and is cleared at the end:

```
read -s "CF_TOKEN?Paste token: "; echo        # zsh; in bash: read -rsp "Paste token: " CF_T
ZONE_ID=<zone id from the domain's Overview page>
DOMAIN=example.com
for v in 'issue pki.goog' 'issue letsencrypt.org' 'issue ssl.com' \
         'issuewild pki.goog' 'issuewild letsencrypt.org'; do
  set -- ${=v}                                  # bash: set -- $v
  printf '%-10s %-16s ' "$1" "$2"
  curl -s -X POST "https://api.cloudflare.com/client/v4/zones/$ZONE_ID/dns_records" \
    -H "Authorization: Bearer $CF_TOKEN" -H "Content-Type: application/json" \
    --data "{\"type\":\"CAA\",\"name\":\"$DOMAIN\",\"data\":{\"flags\":0,\"tag\":\"$1\",\"va
    | grep -o '"success":[a-z]*'
done
unset CF_TOKEN
```

Every line should end in `"success":true`. A record that already exists returns `false`, and you can ignore it.

4. **Delete the token** straight away: API Tokens → ⋯ → Delete.

STEP 4. VERIFY

```
# The records, from the authoritative nameserver and from a public resolver
dig +short CAA example.com @$(dig +short NS example.com | head -1)
dig +short CAA example.com @1.1.1.1

# Every host still serves a valid certificate
for h in example.com www.example.com account.example.com; do
  curl -s -o /dev/null -w "$h %{http_code}\n" -A "Mozilla/5.0" -H "Accept: text/html" "https://
done
```

A 2xx or 3xx on every host means nothing broke. The real test is the next renewal: note each certificate's expiry date and check again after it. Some Shopify hosts return 406 to a bare `curl` with no `Accept` header, which is not an outage.

STEP 5. KEEP IT CURRENT

- **Before adding a custom domain for a new service,** find out which CA it uses and add that CA first.
- **Watch the CT logs** for your domain. Cloudflare's Certificate Transparency Monitoring (SSL/TLS → Edge Certificates) emails you when a certificate is issued for it. crt.sh is the manual check.
- **Record the change** in the key and certificate ledger with the date and who made it, like any other trust decision.

Sources: RFC 8659 — DNS CAA Resource Record; Cloudflare — CAA records; crt.sh Certificate Transparency search.

The number 4, and what row-level controls still protect

What an elliptic-curve key is. An elliptic curve is a fixed, published mathematical curve, with a fixed starting point on it, *G*. "Multiplying" a point by a number means stepping along the curve that many times by a special rule. The rule is easy to run forwards: given a number, finding the point is fast. It is effectively impossible to run backwards: given only the point, there is no shortcut to the number. That one-way step is the whole key pair:

- the **private key** is the number
- the **public key** is the point it lands on

Elliptic-curve keys are much shorter than RSA keys of the same strength, which is why certificates, phones and Ed25519 signatures use them.

eMudhra's keys were elliptic-curve keys, not RSA. An elliptic-curve private key is a single number, *d*. The public key is that number multiplied by a fixed,

published starting point *G*: public key = *d* × *G*. Picked at random from roughly 2^256 possibilities, *d* cannot be worked back from the public key.

For one of eMudhra's keys, *d* was 4. Anyone could compute 1×*G*, 2×*G*, 3×*G* and 4×*G* and match the published key on the fourth try. A researcher recovered all eight keys from the public certificates in under a second, so all of them were tiny. `openssl rand -hex 32` draws *d* from the full 2^256 range. Size was never eMudhra's problem; the randomness was missing.

Why elliptic curves? The business reason. Elliptic-curve keys give the same security as RSA for far less size and work:

	ELLIPTIC CURVE (256-BIT)	RSA OF THE SAME STRENGTH (ABOUT 3,072-BIT)
Public key	32–64 bytes	About 384 bytes
Signature	64 bytes	About 384 bytes
Making a key	Milliseconds, even on a phone	Much slower — two large primes must be found
Signing	Fast	Several times slower
Fits in a QR code or a handle	Yes	No

Smaller and faster means less bandwidth, quicker page loads, longer battery life and lighter servers, on every handshake and every signature.

- **A certificate authority**, eMudhra included, builds an elliptic-curve hierarchy for the usual reasons. eMudhra has not, as far as we have seen, stated its own.
  - Customers want lighter, faster HTTPS.
  - Browsers and TLS 1.3 already favour elliptic curves.
  - A new elliptic-curve root keeps the CA in browser root programs for the next decade as RSA roots age.
  - Everything it issues and revokes gets cheaper.

- **Nostr** chose Bitcoin's own curve (secp256k1) and signatures (BIP-340).
  - Tested libraries, hardware signers and wallets already existed, and the same key sits beside Lightning payments.
  - A 32-byte public key is short enough to be the handle and to fit in a QR code.
  - Relays verify every post, so small, fast signatures keep a network nobody owns cheap to run.
  - A phone makes a key instantly, with no sign-up server.

The curve was not the mistake. Weak randomness breaks RSA just as surely:

- **Debian, 2008:** an OpenSSL bug left only about 32,000 possible keys on Debian systems.
- **"Mining your Ps and Qs", 2012:** researchers found thousands of internet devices whose RSA keys shared primes, which made them computable.

Elliptic curves are slightly less forgiving of carelessness, though. ECDSA leaks the key if a single signature's random number is weak, which is why newer systems prefer EdDSA. The next cost is shared by both: a large quantum computer would break elliptic curves and RSA alike. The post-quantum replacements (ML-DSA for signatures, ML-KEM for key exchange) have much larger keys and signatures, so the size advantage that drove these choices will shrink in the migration.

RSA, elliptic curve and EdDSA — what each one is. RSA and elliptic curve are two kinds of key. EdDSA is not a third kind: it is a way of *signing* with an elliptic-curve key.

	RSA	ELLIPTIC CURVE
The private key is	Two large secret prime numbers	One secret number
The public key is	Their product — a very large number	A point on a published curve, reached by "multiplying" the starting point by that number
Why it is hard to break	Splitting a huge number back into its primes is effectively impossible	Working back from the point to the number is effectively impossible
Size for the same strength	About 3,072 bits	256 bits
What it can do	Sign and encrypt	Sign (ECDSA, EdDSA) and agree a shared key (ECDH)
Where you see it	Older certificates, older SSH keys, many enterprise systems	Most modern HTTPS certificates, phones, Bitcoin, Nostr, eMudhra's hierarchy

SIGNING WITH AN ELLIPTIC-CURVE KEY	ECDSA	EDDSA (ED25519 IS THE COMMON FORM)
Age	Standardised around 2000	Published 2011, standardised 2017
The one-time number in each signature	Must be freshly random. Weak or reused once, and the key can be computed from two signatures	Derived from the key and the message, so a bad random source at signing cannot leak the key
Settings to get wrong	Several curves and options	Essentially one choice per curve
Speed and side channels	Good, with careful implementation	Fast, and designed to resist timing attacks
Where you see it	Most TLS certificates, eMudhra's chain included; Bitcoin before Taproot	`ssh-ed25519` keys, Tor, and CRM Sync's mandates and receipts — the `EdDSA` in the token header the verify plane checks

Nostr uses a third elliptic-curve signature, Schnorr (BIP-340). It is a close relative of EdDSA that also mixes fresh randomness into the one-time number.

In one line each:

- **RSA:** big keys built on prime numbers. Well understood and widely supported, but heavy.
- **Elliptic curve:** small keys built on a curve. The same strength, much lighter.
- **EdDSA:** a modern way to sign with an elliptic-curve key that removes the most common way to lose it.

None of them survives a key made without real randomness — eMudhra's failure, and Debian's RSA failure in 2008 — and a large quantum computer would break all three. The post-quantum replacement for signatures is ML-DSA.

Which layer broke. A certificate authority's key does not encrypt anything of yours. It *vouches* for other people's certificates. Whoever held eMudhra's keys could issue a "valid" certificate for any domain — a Xano instance, a store, pypi.org — and every client that still trusted emSign would accept it.

	THE CA REGISTRY (EMUDHRA)	THE DATA PLANE (XANO BEHIND CLOUDFLARE)
What the key does	Vouches for other people's certificates, for any domain	Encrypts one connection. Xano's certificate secures only Xano
Who vouches for CRM Sync's hosts	Not eMudhra	The Xano instance: Let's Encrypt, chaining to ISRG Root X1, with a 90-day certificate renewed automatically. `crm-sync.dev`: Google Trust Services
Why that is not enough	An attacker does not need your CA. They present a forged certificate to any client that still trusts emSign	The exposure is the *client's* trust store: a laptop, a container, a server — not the host's choice of CA
Scope of a failure	Every site, worldwide, for every client not yet updated	One connection

What still holds if a connection is intercepted. Nothing at the row level fixes a broken CA — updating trust stores does. But row-level design decides how much an intercepted connection can give away:

- **Sealed identity.** The session token is encrypted (JWE), and only the data plane decrypts it. Someone reading the traffic cannot read or alter the claims inside. They can replay the whole token, which is why logout writes it to a denylist checked on every read, and why sessions are short.
- **Row-level authorization.** Every read and write takes the caller's identity from the decrypted token on the server — the `/auth/me` claims and extras — never from a user id the browser sends. A captured session exposes **one person's rows, not the table.**
- **Field-level encryption.** A field encrypted with a key the application holds stays ciphertext even when its row is read.
- **The exception is a master key.** An admin key or a data-plane API key that crosses an intercepted connection has no row limit. That is a layer 1 or layer 6 key in the key layers: rotate it through the ceremony, from a patched machine.

EdDSA vs. token gating — proof is not permission. Two ideas get blurred together:

	EDDSA SIGNATURE (ED25519)	TOKEN GATING
Question it answers	*Who issued this, and has it been changed?*	*Should the holder be let in?*
How	The issuer signs with a private key; anyone checks it with the published public key, offline	The server lets in whoever presents a valid token — possession is the test
What it proves	Authenticity and integrity. A changed character fails the check	Only that someone holds the token
What it does not prove	That the person presenting it is the person it was issued to	That the token was not copied, replayed or stolen
In CRM Sync	Mandates, receipts and certificates, checked on the verify plane against the JWKS	Sessions and entitlements: the server decrypts the token, checks the denylist and expiry, then applies the caller's roles row by row

A signed token used as a gate is still a **bearer** credential: whoever holds it, gets in. So signing is never the whole answer. The gate adds:

- a short expiry
- revocation checked on every read
- an audience, so a token minted for one service is refused by another
- spending caps and scopes on agent mandates
- server-side role checks on each row

Signing makes a token **trustworthy**. Gating decides what that trust is **worth**. The same split runs through web3 "token gating", where holding an NFT opens a door: the chain proves the token is genuine, not that the wallet holding it is the person it was meant for.

Why EdDSA, after eMudhra. eMudhra's chain used ECDSA, the older elliptic-curve signature scheme. ECDSA needs a fresh random number for *every signature*. Reuse one, or let it be guessable, and the private key can be computed from two signatures — a well-known failure. EdDSA derives that

number from the key and the message, so a weak random source at signing time cannot leak the key.

EdDSA would not have saved eMudhra. Their failure was in *making* the key, and every scheme needs real randomness at key generation. That is why CRM Sync's Ed25519 keys come from the platform's cryptographic random source inside the Worker, never from a hand-run script.

Nostr: when the key is the identity

Nostr, the messaging protocol Jack Dorsey championed and funded, uses the same kind of key as eMudhra — elliptic curve — but makes the opposite choice: **the key is the identity and the permission at once.**

- **The account is a key pair** on secp256k1, Bitcoin's curve, with Schnorr signatures.
- **The public key (`npub…`) is the handle.** The private key (`nsec…`) is the password, the account and the signature, all at once.
- **Every post is a signed event.** Relays — the servers — check the signature and pass the event on. No server issues the identity or can take it away.
- **The optional NIP-05 name** (`name@domain`) is only a file on a website that points a name at the key.

	NOSTR	EMUDHRA (A CERTIFICATE AUTHORITY)	CRM SYNC
Who vouches for a key	Nobody — trust comes from who follows it	The CA, for any domain	The issuer's own published JWKS
Who makes the key	The user's app, on the user's device	The CA, in a ceremony	The Worker, from the platform's cryptographic random source
What a signature means	"This key wrote it" — which is also the permission to post	"This CA vouches for this site"	"We issued this and it has not changed" — proof only
Who decides access	Nobody beyond the signature; relays may add their own gates (paid relays, allowlists)	The client's trust store	The server, every time: expiry, revocation, roles per row
Rotation	None built in. A new key is a new identity, without its followers	Revoke and reissue — if clients check	Key ids, end dates and a JWKS edit; old receipts keep verifying
If the private key leaks	Anyone can post as you, permanently; there is no recovery	Fake certificates for any site, until trust stores update	Limited to that key's layer, and rotated through the ceremony
If the private key is lost	The identity is gone	—	Rotate; nothing is lost, because the server holds the records

What Nostr shows about everything above.

- **Same kind of key, opposite outcome: Nostr is randomized where eMudhra was not.** Both use elliptic-curve keys — a private key that is a single number — though not the same curve.
  - **Key creation.** Each Nostr app makes its key on the user's device, normally from the operating system's cryptographic random source, so the number is one of roughly 2^256 — not 4.

    - **Signing.** Nostr's Schnorr signatures (BIP-340) build each signature's one-time number from the key, the message and fresh random bytes. A weak random source at signing time therefore cannot leak the key — the ECDSA failure that eMudhra's chain is exposed to.
    - **The limits.** The protocol cannot check how a key was made. There is no ceremony and no authority, so a badly written app could still produce a weak key, and nothing in the network would notice.
    - **Scale, not kind.** A weak Nostr key exposes one person. A weak CA key exposes every site that CA can vouch for. eMudhra's central ceremony was supposed to be the careful one, and it failed once, for everyone.

  The lesson is the same for both: the type of key was never the problem. What matters is whether the private key came from real randomness, and whether anyone checked.

- **Pasting an `nsec` into apps is the key-ceremony problem.** Every app it is pasted into — including AI tools and browser extensions — can take the identity for good. Nostr's answer is a remote signer (NIP-46, a "bunker"): the key stays in one place, and apps ask it to sign. That is the ceremony's rule — the thing that holds the key is not the thing that asks for its use.
- **Proof is not permission — unless you choose to merge them.** Nostr merges them on purpose, and gains censorship resistance and independence from any platform. The price is a leaked key that cannot be revoked and no row-level limit on what it can do. Commerce cannot pay that price: a stolen customer or admin key has to be stoppable today. So CRM Sync keeps the signature as proof and leaves the decision to a server that can say no.
- **eMudhra touches Nostr only at the edges.** Post signatures do not depend on certificate authorities, so a forged certificate cannot forge a post. But relays are reached over secure WebSockets and NIP-05 names are fetched over

HTTPS. An intercepted connection could fake a name or hide posts — it could not sign as you.

Details are from the Nostr specifications (NIP-01, NIP-05, NIP-19, NIP-46) — see the NIPs repository.

Which custody model fits an agent? Participant-held keys (Nostr) and edge-held keys (CRM Sync) are compared side by side — identity versus authority, rotation after a compromise, and enterprise controls — in Key Custody: Two Ways to Give an Agent a Key.

Non-random keys were never safe — AI only shortens the window

Would eMudhra's keys have been fine before AI? No. A private key of 4 is found by trying 1, 2, 3, 4 — a pocket calculator, not a model. The keys were breakable from the ceremony on 30 April 2025. Every well-known weak-key failure was weak the moment it was made, and humans found each one:

YEAR	INCIDENT	WHAT WAS WRONG	FOUND BY
2008	Debian OpenSSL	A code change left only about 32,000 possible keys	A developer reading the code
2010	Sony PlayStation 3	ECDSA signing reused the same "random" number every time, so the private key could be computed	Hobbyist researchers (fail0verflow)
2012	"Mining your Ps and Qs"	Thousands of devices made RSA keys with too little randomness at boot; many shared primes	Academic researchers scanning the internet
2017	ROCA (CVE-2017-15361)	A smart-card chip's library made RSA keys with a hidden structure — about 750,000 Estonian ID cards among those affected	Academic researchers
2025–26	eMudhra	Eight CA keys, one of them the number 4	An independent researcher checking published CA certificates

What AI changes is the window, not the maths.

- **Searching is cheap now.** Scanning every certificate in the public CA database or the Certificate Transparency logs for weak keys was always possible. It is now cheap, quick and automated, for researchers and attackers alike. AI-assisted tools are already credited with finding flaws such as php2shell.
- **Mistakes no longer sit quietly.** eMudhra's keys sat for about 18 months before anyone looked. Today, expect a mistake like that to be found in days, possibly by someone who will not report it.

So checking has to happen when a key is made, not years later:

- generate from a cryptographic random source
- lint the certificates
- test new keys against known-weak patterns
- monitor Certificate Transparency for your domains
- record key fingerprints, so a duplicate or reused key stands out

Security levels, in one table. Strength is counted in *bits of security*: 128 bits means an attacker needs about 2^128 tries, far beyond any computer that exists. Different key types reach that level at different sizes.

SYSTEM	KEY	SECURITY LEVEL AS DESIGNED	SECURITY LEVEL AS ACTUALLY MADE
eMudhra intermediates	Elliptic curve (ECDSA)	About 128 bits or more	**Effectively none** — all eight recovered in under a second
Nostr accounts	secp256k1, Schnorr (BIP-340)	About 128 bits	About 128 bits **if the app used a proper random source**; nothing in the protocol checks
CRM Sync mandates and receipts	Ed25519 (EdDSA)	About 128 bits	About 128 bits — generated by WebCrypto in the Worker
CRM Sync agent-commerce signing (UCP)	ECDSA P-256	About 128 bits	About 128 bits — generated by WebCrypto; ECDSA because the standard requires it
Google service-account keys	RSA-2048	About 112 bits	About 112 bits — issued by Google
CRM Sync vault and config encryption	AES-256-GCM	256 bits (about 128 against a future quantum computer)	Depends on the master secret it is derived from — see the key layers

Nostr's security level, plainly.

- **On paper it matches CRM Sync's signing:** about 128 bits, on a curve that has protected Bitcoin for years.
- **In practice, the weak points are operational, not mathematical:**
  - each app must use a proper random source, and nothing in the protocol checks
  - there is no rotation or revocation, so one leaked `nsec` is permanent
  - pasting the key into apps exposes it — the remote signer (NIP-46) exists for that reason
- **Message privacy depends on which version an app uses.** Nostr's older direct-message encryption (NIP-04) is deprecated for known weaknesses

and replaced by NIP-44. Apps still on the old scheme protect less than the signatures suggest.

- **The shared limit:** a large quantum computer would break Nostr's keys, eMudhra's and CRM Sync's signing keys alike.

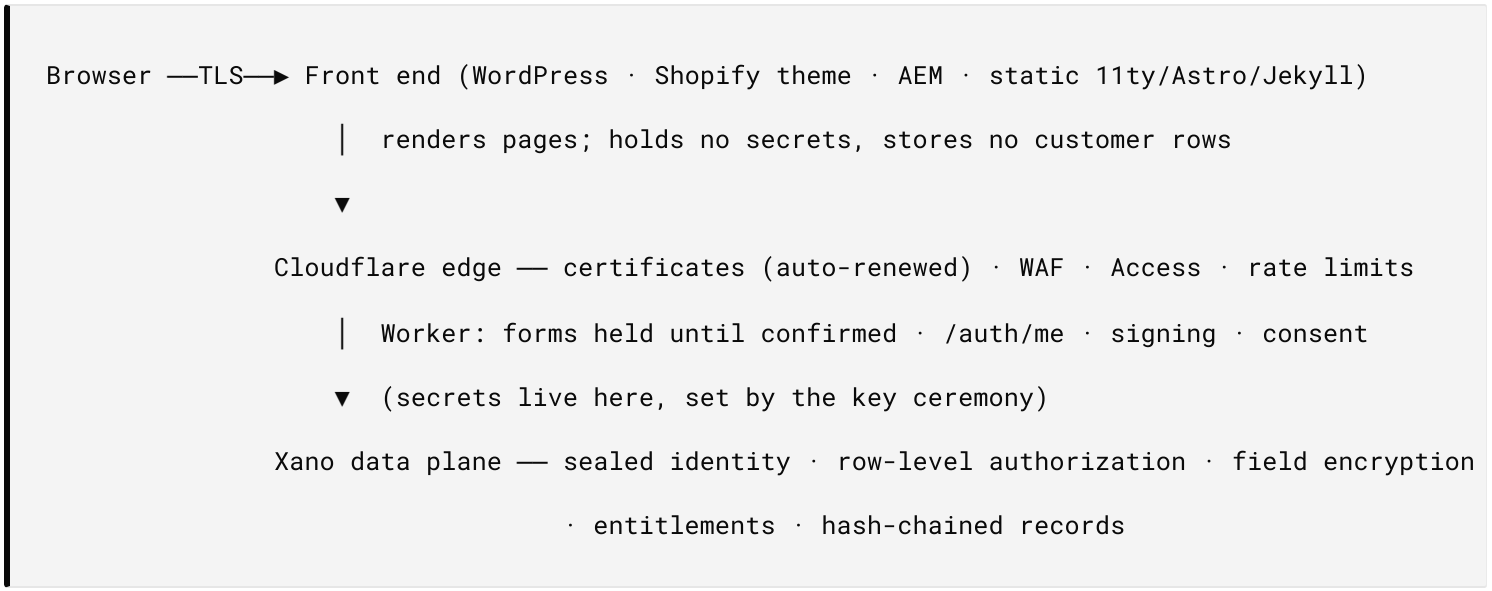
The level a key *could* have is set by its type. The level it *does* have is set by how it was made and how it is kept. eMudhra had the right type and none of the second.

One architecture under any front end, whatever the platform plan

The controls in this article do not belong to WordPress, Shopify, AEM or a static site generator. They sit in two layers the front end calls:

- **Cloudflare at the edge:** certificates, the WAF, Access, and Workers for forms, sign-in, signing and consent.
- **Xano as the data plane:** identity, rows, entitlements and encryption.

The front end renders pages and calls the edge. It never holds a key and never stores a customer row.



FRONT END	STAYS ON THE PLATFORM	MOVES TO CLOUDFLARE + XANO	HOW IT CONNECTS
WordPress	Pages, posts, the theme	Forms, sign-in, customer records, keys. Card checkout goes to Shopify through the Shopify plugin	One script tag and two form attributes; the PHP origin sits behind Cloudflare
Shopify	Catalog, cart, checkout, payments — the PCI scope stays with Shopify	Consent, loyalty and review records, agent mandates, signed receipts	Theme app embed or Storefront Web Components
AEM	Content, components, the DAM	The same as WordPress	A client library or embed in the page component
Static (11ty, Astro, Jekyll, Webflow export)	Built HTML on any host	Everything dynamic	The same script and attributes; nothing to patch on the host

Why the plan or fee does not matter. Shopify Basic or Plus, WordPress.com or self-hosted, an AEM licence or a free GitHub Pages site — the protection is the same, because none of it runs on the front end's plan:

- the key ceremony and key layers apply to secrets the front end never sees
- forms are held until the inbox confirms, at the edge
- identity is sealed and rows are authorized in the data plane
- certificates renew at the edge

A platform upgrade buys features, not these controls. Changing platform does not lose them either. The front end can be replaced, and the same edge and data plane carry over unchanged, along with their records.

What still depends on the front end:

- **Its own runtime must be patched.** php2shell on a WordPress origin is still the host's to fix, and the edge only buys time.

- **Its theme must not leak.** No client-side redirects to other domains, and no keys in theme code.

The working example — one theme, one form, one sign-in across platforms — is in the safer customer option and secure forms on any front end.

The key ceremony: a person makes the key, the AI never sees it

eMudhra shows what goes wrong when keys are made badly. LiteLLM shows where good keys leak: terminal output, shell history, the clipboard, `.env` files and chat transcripts. Today those are also exactly what an AI coding assistant reads. A **key ceremony** is the routine that keeps a secret out of all of them.

The rule is simple: **a person runs the step that creates the secret, and the AI only prepares, checks and records.** The secret goes from the generator straight into the secrets store, through a pipe or a prompt. It is never printed, never pasted into a chat, and never written into a command line.

WHO	DOES	NEVER SEES
The key holder (a security role, not the developer who asked for the change)	Runs the command that creates and stores the secret, in their own terminal	—
The AI assistant	Writes the steps, checks the result by fingerprint and behaviour, files the record	The secret
The record	Who, which secret, when, its fingerprint, how it was checked	The secret

Directions — OpenSSL and Wrangler, interactive. Run these from the Worker's folder.

Before you start: patch the machine the ceremony runs on. `wrangler secret put` sends the new key to Cloudflare over HTTPS, and Wrangler runs on Node. If that machine's Node still trusts the eMudhra chains, a fresh key could travel

over an intercepted connection. Use Node 22.23.4, 24.22.1 or 26.12.1 or later. If Python is on the machine, follow Python and Docker after eMudhra first.

1. **Open your own terminal window, not the AI's.** If you work in an AI coding tool, do not run ceremony commands through it. That includes shortcuts that run a command inside the session, because their output becomes part of the conversation.
2. **Generate and store in one step, so the value is never displayed:**

   ```
   openssl rand -hex 32 | npx wrangler secret put ADMIN_KEY --config wrangler.toml --env=""
   ```

   Wrangler reads the value from the pipe. Nothing is echoed, and the shell history holds the command but no secret.

   How big is it? `rand -hex 32` means 32 random bytes: 256 bits, written as 64 hexadecimal characters (0–9, a–f). That is 2^256 possible keys, a number 78 digits long. Guessing at a trillion tries a second would take far longer than the age of the universe.

   Why it is safer than eMudhra's keys. eMudhra's keys were not short. They were made badly. A wrap-and-unwrap step in the ceremony produced keys with so little real randomness that they could be worked out from the public certificates in under a second. Length is only worth anything if every bit is random. `openssl rand` takes its bytes straight from the operating system's cryptographic random source, with no extra step between the source and the key. The scale is different too. This secret guards one Worker, nobody else trusts it, and replacing it is one command. A CA key is trusted by every browser, and replacing it took four browser vendors and a root removal.
3. **For a secret someone else issued** (an OAuth client secret, a payment-provider key), use the interactive prompt:

```
npx wrangler secret put GOOGLE_CLIENT_SECRET --config wrangler.toml --env=""
```

Run the command **first**. Then copy the secret from the provider's dashboard and paste it at the "Enter a secret value" prompt. Done in the other order, copying the command overwrites the secret on the clipboard, and the command text is stored as the secret.

4. **When one secret must go to two systems** — say a Xano environment variable and the Worker — write it to a file only you can read, and feed both from that file:

```
( umask 077; openssl rand -hex 32 | tr -d '\n' > ~/.crm-ingest-key )
npx wrangler secret put INGEST_KEY --config wrangler.toml --env="" < ~/.crm-ingest-key
pbcopy < ~/.crm-ingest-key     # paste into Xano, then clear the clipboard:
pbcopy < /dev/null
```

5. **Check by fingerprint, never by value.** The AI can be shown a fingerprint: the first eight characters of a hash, which say nothing usable about a 256-bit key.

```
shasum -a 256 ~/.crm-ingest-key | cut -c1-8
npx wrangler secret list --config wrangler.toml --env=""   # names only, never values
```

Then check behaviour. The old key is refused, and the new key works. Any check refers to the file (`$(cat ~/.crm-ingest-key)`) and never repeats the value, so the AI's command and output both stay clean.

6. **Record it, then delete the file.** File the record: who ran it, which secret, when, the fingerprint and the check result. Then delete any file you made.

Signing keys are not made this way at all. The Ed25519 keys behind mandates and receipts are generated by WebCrypto inside the Worker. Rotation is a server

action. The private half never exists on a laptop, and only the public half is ever published. OpenSSL and Wrangler are for **shared secrets**: admin keys, webhook secrets, API keys from other providers.

You manage the keys you hold — create, rotate, revoke — and the rest of the plane from one console: Configuration System. Every change lands in the audit ledger. The full ceremony, with roles and the record format, is in Rotate a Key in Three Steps and the Key Management Lifecycle.

Dev, Stage and Prod: a separate ceremony for each, least privilege everywhere

A key that works in two environments is one leak away from both. Each environment gets **its own keys, made in its own ceremony**, and no value is ever copied from one to another. The fingerprints prove it: if Stage and Prod show the same eight characters, something was copied. The full Dev → Stage → Prod workflow, with roles for consulting teams and stakeholders, is in the Key Management Lifecycle.

ENVIRONMENT	WHERE IT RUNS	KEYS	WHO CAN SET THEM	COMMAND
Dev	The developer's machine (`wrangler dev`)	Throwaway values in an uncommitted `.dev.vars` ; no real customer or provider keys	The developer	`openssl rand -hex 16` into `.dev.vars`
Stage	A separate Worker, `[env.staging]` , with its own storage	Generated for Stage only; test-mode provider keys	The release engineer	`openssl rand -hex 32 | npx wrangler secret put NAME --config wrangler.toml --env=staging`
Prod	The production Worker	Generated for Prod only; live provider keys	The key holder, from an approved machine on the company network	`openssl rand -hex 32 | npx wrangler secret put NAME --config wrangler.toml --env=""`

Passing `--env` explicitly matters. With more than one environment defined, a bare Wrangler command falls back to the top level, which is right only by accident.

Least privilege: each person, machine and test gets the smallest key that does its job.

- **Production secrets are set on-premises only.** The Cloudflare API token that can write Prod secrets belongs to the key holder. It is scoped to the one account and the Worker permissions it needs, and it can be locked to the company network's addresses with Cloudflare's client-IP filter on API tokens. Admin paths sit behind Cloudflare Access with a device check. A token copied off the network does not work.
- **Supply-chain tests never see a secret.** Dependency scans, crate audits and the static test harness run on every pull request, so they treat every change

as untrusted. CRM Sync's pipeline shows the pattern:

  - the job token can only read the repository
  - actions are pinned to commit hashes, not movable tags
  - installs run with `--ignore-scripts`
  - the contract tests touch only public and fail-closed paths, so they are safe on pull requests from outside forks

  The one credential the test job holds is a single-repository, read-only token for a private test dependency.

- **Deploy is a separate job with separate rules.** It runs only on a merge to `main`, never on a pull request, and it goes through the same deploy guard and test gate as a deploy from a laptop. Worker secrets are not stored in CI at all. They are set by the ceremony, so a poisoned dependency in CI has nothing to steal.
- **Reviewers approve; they do not hold keys.** Design, content and compliance roles sign off on what ships. Only the release engineer, whose role carries the release permission, promotes it. The agency or consulting team gets a rotatable key, never the root key and never the command line.

Trust setup: decide once, per environment, what each system will believe. Most of the breaches on the timeline were failures of trust, not of encryption. Write the trust list down and keep it under version control:

WHAT IS TRUSTED	SET WHERE	CHANGES ONLY BY
Signing keys that verify mandates and receipts	The published JWKS, with key ids and end dates	Rotation through the ceremony
Certificate roots a program accepts	The trust store in each runtime and container, listed in the SBOM	An updated runtime or bundle, recorded in the SBOM — how the eMudhra roots get removed
Packages and crates	Lockfiles, hash-pinned installs, and `cargo vet` against named published audits	A reviewed audit, never a wildcard
Build tools and CI actions	Commit-hash pins and a read-only job token	A reviewed pull request
People and agents	Roles granted by invitation; agent mandates with caps and expiry	An entitlement change on the ledger
Networks	Client-IP filters on API tokens; Cloudflare Access on admin paths	The key holder

The configurator (Configuration System) shows which environment you are in and which keys you hold. Environment rules, the key-handoff matrix and the RACI are in the Key Management Lifecycle.

Key layers: what each key protects

Not every key is equal. Each layer protects something different, is held by someone different, and costs something different to replace. Knowing which layer a key belongs to tells you who may hold it and what to do the day it leaks.

LAYER	EXAMPLE IN CRM SYNC	WHAT IT PROTECTS	WHO HOLDS IT	IF IT LEAKS
1 · Platform root	`ADMIN_KEY`, and the Cloudflare API token that deploys and sets secrets	Everything: configuration, every tenant, the ability to change the other keys	The key holder only. Never an agency, never CI	Rotate at once through the ceremony, then rotate every key below it
2 · Rotatable admin	A rotatable admin key issued alongside the root	Day-to-day admin work, without exposing the root	The operations team	Rotate it. The root and other teams are untouched
3 · Tenant	`crm_t_` tenant tokens	One store's data and settings — nothing else	That store's team, or its agency	Revoke that one token; other tenants never notice
4 · Sessions	`JWT_SECRET` (HMAC-signed session tokens)	Signed-in sessions: who is logged in right now	The platform, as a secret, never a person	Rotate; every session ends and people sign in again. **Today it also does a second job** — see below
5 · Signing	Ed25519 (EdDSA) keys for entitlements, mandates and receipts; ECDSA P-256 keys for agent-commerce (UCP) request signing, which the UCP standard requires	Receipts, mandates and certificates anyone can check offline; agent requests other platforms can verify	Generated inside the Worker; only the public halves are published (`/.well-known/jwks.json`, `/.well-known/ucp`)	Rotate. Past receipts keep verifying until the old key's end date; new ones use the new key
6 · Provider	Shopify Admin token, Xano API key, Google client secret, payment keys	Access to systems someone else runs	The platform as secrets, set from the provider's dashboard by the key holder	Rotate at the provider first, then store the new value with the ceremony

A correction we found in our own setup. The vault and the encrypted configuration fields are meant to have their own master secret, `ENVELOPE_MASTER_KEY`. When it is not set, the code falls back to `JWT_SECRET` — and in CRM Sync's production it is not set. So one secret is doing two layers' jobs:

- it signs sessions (layer 4)
- it is the root of the encryption keys for vaulted firmware and tenant configuration

Rotating `JWT_SECRET` today would not only sign everyone out; it would also leave every vaulted image and encrypted configuration field unreadable until they are re-wrapped. That is exactly the "one key spanning layers" problem instruction 2 below warns about.

The fix is a planned migration, not a one-line change:

1. Create `ENVELOPE_MASTER_KEY` through the ceremony.
2. Re-wrap every existing envelope under it.
3. Only then leave `JWT_SECRET` to sessions alone.

Setting the new secret without step 2 would lock the existing data at once, because the code would immediately start deriving keys from it. Until the migration is done, treat `JWT_SECRET` as a layer 1 key.

Instructions — run the layers in this order:

1. **List every key you have and give it a layer.** If you cannot say which layer a key protects, it is probably layer 1 by accident. An "all-access" token used for convenience is the common case.
2. **Split anything that spans layers.** One key doing root work and tenant work means a tenant leak becomes a platform leak.
3. **Give each layer its own holder.** Agencies and contractors get layer 2 or 3, never layer 1. Reviewers and AI assistants get none.

4. **Give each layer its own environment copy.** Dev, Stage and Prod never share a value at any layer.
5. **Set a rotation date per layer** — the root on staff change and at least yearly, the others every 90 days and on any incident — and record each rotation by fingerprint.

Without Cloudflare

The ceremony does not need Cloudflare. Wrangler is only Cloudflare's way into its secrets store, and the rule is the same everywhere: **create the secret with OpenSSL and send it straight into the store, through a pipe or a file only you can read.**

PLATFORM	SECRETS STORE	GENERATE AND STORE, NOTHING SHOWN
Google Cloud	Secret Manager	`openssl rand -hex 32 | gcloud secrets versions add NAME --data-file=-`
AWS	Secrets Manager	`( umask 077; openssl rand -hex 32 > ~/.k )`, then `aws secretsmanager put-secret-value --secret-id NAME --secret-string file://$HOME/.k`, then delete `~/.k`
Azure	Key Vault	Same file approach: `az keyvault secret set --vault-name V --name NAME --file ~/.k`
HashiCorp Vault	KV store	`openssl rand -hex 32 | vault kv put secret/app api_key=-`
GitHub Actions	Repository or environment secrets	`openssl rand -hex 32 | gh secret set NAME --env production`
Kubernetes	Secrets, ideally through an external secrets operator	`openssl rand -hex 32 | kubectl create secret generic app --from-file=api_key=/dev/stdin`
Vercel	Environment variables	`openssl rand -hex 32 | vercel env add NAME production`
A server you control	A file only the app's user can read, outside the web root	Over SSH: `( umask 077; openssl rand -hex 32 > /etc/app/api_key )`

Check each command against the provider's current documentation before relying on it.

Shared hosting is the exception. cPanel-style PHP hosting usually has no secrets store at all. Keys end up in `wp-config.php` or a `.env` inside the web root, readable by anything that runs there — including a php2shell exploit. You cannot run a proper ceremony on it. Move secrets outside the web root with `chmod 600` today, and plan the move off it.

The other Cloudflare pieces in this article have equivalents:

THIS ARTICLE USES	WITHOUT CLOUDFLARE
Edge certificates issued and renewed automatically	Let's Encrypt via certbot, Caddy's built-in HTTPS, AWS Certificate Manager, Google-managed certificates
Cloudflare Access on admin paths	Tailscale or a VPN, Google IAP, AWS Verified Access, an IP allowlist
Client-IP filter on the API token	AWS IAM `aws:SourceIp` conditions, Google VPC Service Controls, Azure Conditional Access
WAF rules for php2shell	AWS WAF, Google Cloud Armor, Azure WAF — patching PHP is still the only complete fix

To use the Cloudflare route instead, the free plan is enough to start:

1. **Sign up at dash.cloudflare.com/sign-up.** Workers are on the free plan and run on a `workers.dev` address, so you do not need a domain.
2. **Install Wrangler in your project:** `npm install --save-dev wrangler`. Use Node 24.22.1 or later, which rejects the eMudhra chains.
3. **Sign in from your own terminal:** `npx wrangler login`. This opens the browser and stores a login token on the machine. On a shared machine, run `npx wrangler logout` after the ceremony. For production, prefer an API token scoped to the one account, locked to your network's addresses.
4. **Run the ceremony** above, one environment at a time.

To manage the keys you hold in CRM Sync — create, rotate, revoke — open the Configuration System.

TLS automation: replacing a certificate is a renewal, not a project

Browsers received the eMudhra block automatically. Much other software did not:

- Go binaries, which need a rebuild with a fixed Go
- anything using Python's `certifi` bundle

- the curl that ships with macOS
- older `cloudflared`
- origin servers running their own mutual TLS
- containers with a baked-in trust store

Two habits close that gap:

- **Let the platform issue and renew certificates.** On Cloudflare, edge certificates are issued and renewed automatically. A certificate that chains to a distrusted root is replaced by the next renewal, not by a ticket. A site that buys and installs certificates by hand has until 27 October to get off the emSign hierarchy.
- **Treat the trust store as a dependency, and list it in the SBOM.** `certifi`, `webpki-roots`, the Node and Go versions and the container base image all decide which roots a program trusts. When they are listed, "who still trusts the bad root" is a search across releases.

Python and Docker after eMudhra: what to do this week

Python is the slowest runtime to recover from eMudhra, for a structural reason: **pip carries its own copy of `certifi` and always trusts it, on top of the operating system's list.** There is no setting to turn it off, and `--cert`, `PIP_CERT` and `REQUESTS_CA_BUNDLE` only *add* certificates. A fully patched operating system still leaves an old pip trusting the bad chain. The risk is an intercepted connection serving a fake PyPI, and the only fix is a new pip.

TOOL	FIXED VERSION	NOTES
pip	**26.2.2** or later	The one that matters most: it brings its own certificates
certifi	**2026.10.19**	Removes the emSign roots (advisory GHSA-6q48-6c7w-9gw2)
uv	**0.13.4** or later	Advisory GHSA-hq85-5cpj-mr62; run with system certificates
sigstore, conda, pixi	4.5.1 · 26.9.4 · 0.82.1	—
Node	22.23.4 · 24.22.1 · 26.12.1	Same releases as the Brotli fix

Versions are from the Python packaging announcement on discuss.python.org.

Upgrade pip from the pinned wheel, not with a bare `pip install -U pip`. An unpatched pip fetching its own replacement is exactly the connection an attacker would intercept. The announcement gives a `pip install` command naming the 26.2.2 wheel by its SHA-256 hash, so a substituted file is refused. Then upgrade `certifi` and `uv`. This is the hash-pinned install from the supply-chain table, doing the job it exists for.

Every virtual environment, not just the system Python. Each venv has its own pip. A **new** venv still gets the old bundled pip from `ensurepip` until CPython ships its fix (GH-159910). So upgrade pip right after creating any venv, and in CI after every `python -m venv`.

Docker: a patched host does not patch an image. The certificates are baked in at build time.

- **Official `python` images:** the maintainers rebuilt them on 21 October (docker-library/python #1142), with the expedited 3.13.17, 3.14.9 and 3.15.1 releases. The **3.11 and 3.12 images keep an older pip**, because upstream does not change pip in those security releases. Add the pinned-wheel pip upgrade to your Dockerfile, or move to 3.13 or later.

- **Rebuild without the cache**, so the new base and certificates are actually used: `docker build --pull --no-cache`. A cached layer quietly keeps the old trust store.
- **Node and Go images on Debian** also need `ca-certificates` updated inside the image, so curl, wget and git stop trusting the chain.
- **List the trust store in the SBOM** (pip, certifi, `ca-certificates`, base-image digest), so "which images still trust emSign" is a search, not an audit.

And the LiteLLM root cause shows why the key ceremony keeps the AI out. On 22 October LiteLLM disclosed how 1.107.0 was published:

- **The tokens were in logs.** An internet-exposed internal LiteLLM deployment held GitHub CLI OAuth tokens. LiteLLM says an AI coding tool's response bodies can carry the user's GitHub CLI token, and those bodies were logged.
- **The attacker used them to publish.** With three developers' tokens, including one with workflow rights and an organisation owner's token used to switch off branch protection, the attacker pushed the release through a lookalike repository. It was signed and published like a real release.
- **The stolen cloud keys were used within hours.** Stolen AWS credentials were being tested about four and a half hours later.
- **Release rights had just been widened.** LiteLLM had temporarily expanded them for two engineers two days earlier.

Every link in that chain is covered in this article:

- AI traffic carries secrets, so gateway logs must not store request and response bodies.
- Developer tokens should be scoped to one repository and to read access unless a release needs more.
- Owner tokens should not sit on everyday machines.
- Release rights widened "temporarily" need an end date, like any other key.

The order: fix trust first, then rotate keys

New keys do not fix eMudhra. The two problems sit at different layers:

	WHAT IT PROTECTS	WHAT FIXES IT
eMudhra	Whether a computer can tell the real pypi.org — or any site — from a fake one	Updating what the software trusts: pip 26.2.2, certifi, uv, Node, `ca-certificates`, rebuilt images
Your keys (Wrangler secrets)	Your own secrets: admin keys, webhook secrets, provider API keys	The key ceremony

A forged pypi.org is accepted before any of your keys is involved, so rotating them cannot stop it. The order:

1. **Fix trust first.** Update Node, pip (from the hash-pinned wheel), certifi and uv on every machine and in every venv, and rebuild images. This is the eMudhra fix.
2. **Then ask whether anything leaked while you were exposed.** If a `pip install`, a `wrangler secret put` or a deploy ran over an untrusted network while the old trust was in place, a secret could have crossed an intercepted connection. Rotate those secrets with the ceremony, **from the patched machine**.
3. **If nothing points to exposure, rotate on the normal schedule.** eMudhra alone does not require new keys.

The ceremony is the clean-up and the habit, not the cure. Run on an unpatched machine, it would send fresh keys over the same untrusted connection it is meant to protect.

Clean up after exposure: rebuild, don't purge

A package downloaded through an intercepted connection has the right name and the right version, and nothing on disk records that its download was faked. So you cannot find the bad ones by looking, and uninstalling packages

one at a time does not work. **Rebuild instead, from hashes recorded before the exposure.**

1. **Fix pip first**, as above: 26.2.2 from the hash-pinned wheel, then certifi and uv, on a network you trust. A new Python alone does not fix existing environments, because every venv keeps its own pip.
2. **Delete each virtual environment and recreate it from a lockfile with hashes:** `pip install --require-hashes -r requirements.txt`, or `uv sync` from a `uv.lock`. Every package is checked against its recorded hash, and a substituted file is refused. No lockfile yet? Generate one with the patched pip, then rebuild from it.
3. **Clear every cache a tampered file could hide in**, or the rebuild reuses it:
   - pip: `pip cache purge`
   - uv: `uv cache clean`
   - Docker: `docker build --pull --no-cache`
   - any shared wheelhouse or internal package mirror
4. **Rotate secrets only where there is a reason.** A malicious package runs code while it installs, so removing it afterwards does not undo what it read. If installs ran over an untrusted network while the old pip was in place, rotate what those environments could reach, using the ceremony and the patched machine. If every install ran on networks you trust, rebuilding is enough. No attack has been reported.
5. **Scan what you rebuilt** against published advisories: `pip-audit`, or `osv-scanner` across lockfiles and images. A scanner finds known-bad versions, not swapped files. Steps 2 and 3 handle swapped files.
6. **Re-certify what you ship.** Upload each rebuilt image to the firmware vault: it gets a new digest, a new certificate and a new SBOM link. The vault's ledger then lists who downloaded the earlier image and when, which is the list a

customer notice or a Cyber Resilience Act report needs. Partners confirm the new image on the verify plane.

The exception is LiteLLM 1.107.0. That release was malicious, not merely exposed. Anywhere it was installed:

- treat the environment as compromised
- do not start Python there to clean up, because starting Python re-runs the payload
- rebuild from 1.106.2 pinned by digest
- rotate everything that environment could reach

Does `cargo vet` help? For Rust, yes, but at a different layer from eMudhra:

- **Against an intercepted download:** Cargo already checks every crate against the checksum in the committed `Cargo.lock`. A Rust build from a lockfile was protected the way a hash-pinned pip install is. Keep the lockfile committed and build with `--locked`.
- **Against a poisoned release** — the LiteLLM kind, published by a stolen account: this is what `cargo vet` is for. CI fails on any crate version that no trusted, published audit covers, so a hijacked new release is stopped before anyone has reported it. Neither checksums nor scanners can do that, because the bad release is "genuine" when it ships.
- **For eMudhra itself:** Rust programs that use `webpki-roots` need its fixed release and a rebuild. `cargo update -p webpki-roots`, then `cargo vet` asks for that new version to be audited or certified before it ships. That is the review step working as intended.

Python has no direct equivalent of `cargo vet` yet. The nearest substitutes:

- hash-pinned lockfiles
- a delay before adopting new versions
- review of each dependency change in the pull request

That is the Python row of the supply-chain table above.

AI binding: the agent proves its permission on every call

An AI agent in this architecture acts under a signed mandate: who may act, on what, up to what amount, and until when. The mandate is checked against the published key on every call, so:

- an agent holding a mandate signed by a retired key is refused
- a mandate altered after signing fails verification
- a stolen environment yields keys that have already rotated

Bound tasks and TLS hops are covered in Identity, data and AI tasks. Signing is what lets a third party — an auditor, a partner platform, Google — check that work without trusting us.

Records on a timeline that scales out

Every grant, check, firmware download and consent change is appended to a hash-chained ledger. The ten requirements that ledger meets, the rotation calendar, and how erasure requests are handled without breaking the chain are in the AI Trust Framework Requirements. Each entry locks in the one before it, so history can be visibly broken but never quietly edited. The ledger is not one long chain. **Each tenant and each stream is its own thread**: firmware for one product, mandates for one agent, consent for one store.

- **Threads do not wait on each other.** A new store or product line adds a thread, not contention on a shared one, so the record scales horizontally.
- **Order within a thread is set by a sequence number**, not a clock. Two simultaneous writes cannot be mistaken for tampering.

On a breach day, the incident dates in the table above are laid over those threads. "Everything signed by key X between 30 April 2025 and 17 October 2026" or "every image served with an SBOM listing `certifi` older than the fix"

becomes a filter, not an investigation. To see one session's thread across consent, identity, purchases and engagement, open **See a session's story** on the verify plane.

The benefit, control by control

CONTROL	BREACHES IT ANSWERS	BUSINESS BENEFIT
SBOM for every release (CycloneDX or SPDX)	php2shell, Node, LiteLLM, eMudhra trust bundles	"Are we affected?" answered in minutes, and the record EU Cyber Resilience Act reporting asks for
Signed firmware and build images in a vault	A swapped or poisoned image in distribution	Distributors and devices check the hash against the certificate; a leak is stopped by one rotation, with no recall
Signing keys with key ids, rotation and end dates	eMudhra, the LiteLLM key theft	A leaked key's useful life ends at rotation; receipts it already signed keep verifying
Automated TLS issuance, with the trust store in the SBOM	eMudhra	A distrusted chain is replaced at renewal, and lagging runtimes are found by search
AI binding to signed mandates	Agent overreach, stolen agent credentials	Every agent action can be checked by a third party without trusting the operator
Threaded, hash-chained ledger	All of them	A timeline that grows with the business and answers incident questions by filter

What signed records do not do. An SBOM does not patch anything. A signature proves who sealed a file, not that the file is safe. A ledger shows that history was broken; it does not prevent the break. Developer machines and CI runners — where Node, Wrangler and Python run — still need the runtime updates in the timeline. More background: What Is an SBOM? Who Uses This? Everyone. and Firmware, SBOM & the Cyber Resilience Act.

Its role in AI-first resilience: for QA and growth marketing teams

AI agents and answer engines now act on a store's behalf and decide what customers see. A journey that quietly breaks — a consent banner, a checkout step, a tracking tag, a product feed — costs sales before anyone notices. The verify plane gives the two teams who catch those breaks the same signed record to work from.

QA and testing teams

CHALLENGE	SOLUTION	BUSINESS OUTCOME
Release sign-off waits on manual checks across regions, browsers and app versions	Every test session leaves a signed record of each consent, identity, cart and order step, open on the verify plane as **See a session's story**	Faster sign-off, with evidence attached instead of screenshots
"It passed on my machine": a failure can't be reproduced or proven	The session's thread shows exactly which step changed, in order, and cannot be edited after the fact	Defects are reproduced from the record, not argued over
AI agents place orders and refunds faster than anyone can watch	Each agent action carries a signed mandate with a scope, a limit and an end date; tests confirm an agent is refused outside it	Agent behaviour is tested like any other feature before it reaches customers
A dependency or certificate change — like eMudhra — breaks a journey after release	Trust stores and packages are listed in the SBOM, so a test run names the release that changed	Regressions are traced to a release in minutes

AEO and tracking growth marketing teams

CHALLENGE	SOLUTION	BUSINESS OUTCOME
Tracking drifts after site changes, and the report is wrong before anyone notices	Each event carries its consent state and is checked against the signed record	Numbers the team can defend in a budget meeting
Consent rules remove data, and nobody can tell lost data from broken tags	The record shows which events were withheld by consent and which never fired	Consent loss is measured, not mistaken for a tracking bug
AI answer engines cite competitors, or show stale prices and stock	Product, price and review data go out as structured data and through the Merchant API from one source of truth	Accurate answers in AI search, and more citations
Attribution stops at the first redirect or app hand-off	One session thread runs from first touch through consent, sign-in and order	Spend follows what actually converts



No silver bullet: rotate, randomise, look forward

No single control in this article would have stopped every incident in it. TLS did not stop the checkout hijack. MFA did not stop nOAuth. A WAF does not stop a poisoned package, and a package audit does not stop an unverified sign-up. What holds is layers that change faster than an attacker can map them — so the defence has to move on a schedule, and be tested in ways the attacker cannot predict.

PRACTICE	WHAT IT TAKES AWAY FROM THE ATTACKER	CADENCE
Rotate every credential — on a schedule, and immediately on any incident, through a recorded key ceremony	The useful life of a stolen key	Every 90 days, and on event
Short-lived sessions, revocation checked on every read	Replaying a captured session	Every request
Keys generated per deployment, never copied between environments	One leak opening every environment	Every deployment
Unguessable identifiers — random, not sequential — in anything a public response returns	Enumerating accounts, orders and records by counting	Always
A fresh Content Security Policy nonce per response	Reusing an injected script	Every response
Testing the attacker cannot predict: rotating penetration-test scope, AI-assisted scanning for chained attacks, canary credentials that alert the moment anyone uses them	Mapping a defence that never changes	Continuous
A delay and an audit gate on every dependency	The first hours of a malicious release	Every update

Build to the standard that is coming, not the one being enforced today. Every requirement in the table at the top of this article was announced years before it was enforced, and the merchants hurt worst in October 2026 were the ones still meeting the old version. The ones already on the calendar: the Content API for Shopping shut down on 18 August 2026; EU AI Act transparency obligations apply from 2 August 2026; the EU Cyber Resilience Act's reporting obligations began on 11 September 2026, with full obligations from 11 December 2027; GS1 Sunrise 2027 moves retail point of sale to 2D barcodes. Each date, with its source, is in The Compliance Calendar.

What it costs

COST	WHAT IT MEANS	SEVERITY
Rework of vendor plugins	Loyalty, upsell and options apps are rebuilt as Functions, Workers and a ledger, not configured	High — weeks per plugin, not hours
A monthly bill above shared hosting	A private VM, Cloudflare plans or managed Kubernetes cost more than a shared box	Low to high by tier; get current quotes
EmDash maturity	A v0.1.0 preview with a near-empty plugin ecosystem; WordPress's co-founder argues the isolation only holds on Cloudflare	Medium — pilot only
MySQL has no row-level security	WooCommerce and Magento data needs a move to Postgres or filtered views	Medium
New dependency ecosystems	Rust, Python and npm each need their own gate	Medium — the gates are the cost
Familiar tooling retired	Teams lose cPanel, FTP and plugin-store installs	Real, and the point

> **What this does not claim.** It is not a certification and not legal advice. Delisting counts come from public trackers, not Shopify. "Unconfirmed" means exactly that. Prices are deliberately absent: they vary by provider, region and traffic.

What to do next

Ordered by exposure — card data first.

1. **Payment pages.** Inventory every script, remove every client-side redirect, set the CSP, run a private-window test checkout. *Owner: store owner with the developer.*

2. **Delisted and hijack-listed apps.** Remove or replace; revoke their Shopify scopes after cutover. *Owner: store owner.*
3. **PHP runtime.** Patch for php2shell and keep the WAF rules in Block. *Owner: hosting or DevOps.*
4. **Origin.** Cloudflare in front, Full (strict), Access on every admin path, tunnel, inbound ports closed. *Owner: DevOps.*
5. **Containers.** One pinned image per app, secrets injected at runtime. *Owner: developer.*
6. **Identity and data.** `/auth/me` sidecar; a database role per service; RLS or filtered views; personal fields encrypted with a key generated for this deployment. *Owner: developer with compliance.*
7. **Checkout logic.** Scripts and checkout.liquid patterns rebuilt as Rust Shopify Functions. *Owner: Shopify developer.*
8. **Google.** Merchant API migration with loyalty, reviews, comparison and conversions in the same pass. *Owner: marketing operations with the developer.*
9. **Content.** Static publish first, EmDash pilot second. *Owner: content lead.*
0. **Records.** Keep an SBOM for every release, move firmware and build images from storage links into the firmware vault, and give partners and auditors the verify plane instead of a support inbox. *Owner: developer with compliance.*
1. **Decommission** the shared VM after the tunnel has carried production traffic for a full billing cycle. *Owner: DevOps.*

Evidence basis: public vendor, platform and standards documentation linked inline, read on 22 October 2026. Not a certification.
